

SAMSUNG

Samsung Innovation Campus

| Trí tuệ nhân tạo

Together for Tomorrow!
Enabling People

Education for Future Generations

Chương 6.

Học máy 2

- Học không giám sát

Trí Tuệ nhân tạo

Mô tả chương học

📌 Mục tiêu

- ✓ Có thể áp dụng các kỹ thuật học máy cần thiết theo kế hoạch phân tích đã thiết lập.
- ✓ Có thể áp dụng các kỹ thuật học không giám sát khác nhau để nén dữ liệu thông qua việc giảm chiều dữ liệu có kích thước cao thành dữ liệu có kích thước thấp và trực quan hóa dữ liệu bằng cách giảm kích thước.
- ✓ Có thể giải quyết các vấn đề bằng cách áp dụng các kỹ thuật phân cụm khác nhau để phân đoạn chủ đề.
- ✓ Có thể thực hiện phân rã ma trận, phân tích thành phần chính và các ứng dụng.

📌 Nội dung

- ✓ Bài 1. Thuật toán học máy không giám sát
- ✓ Bài 2. Phân cụm phân cấp
- ✓ Bài 3. Phân cụm không phân cấp
- ✓ Bài 4. Mô hình hệ số tuyến tính để giảm chiều

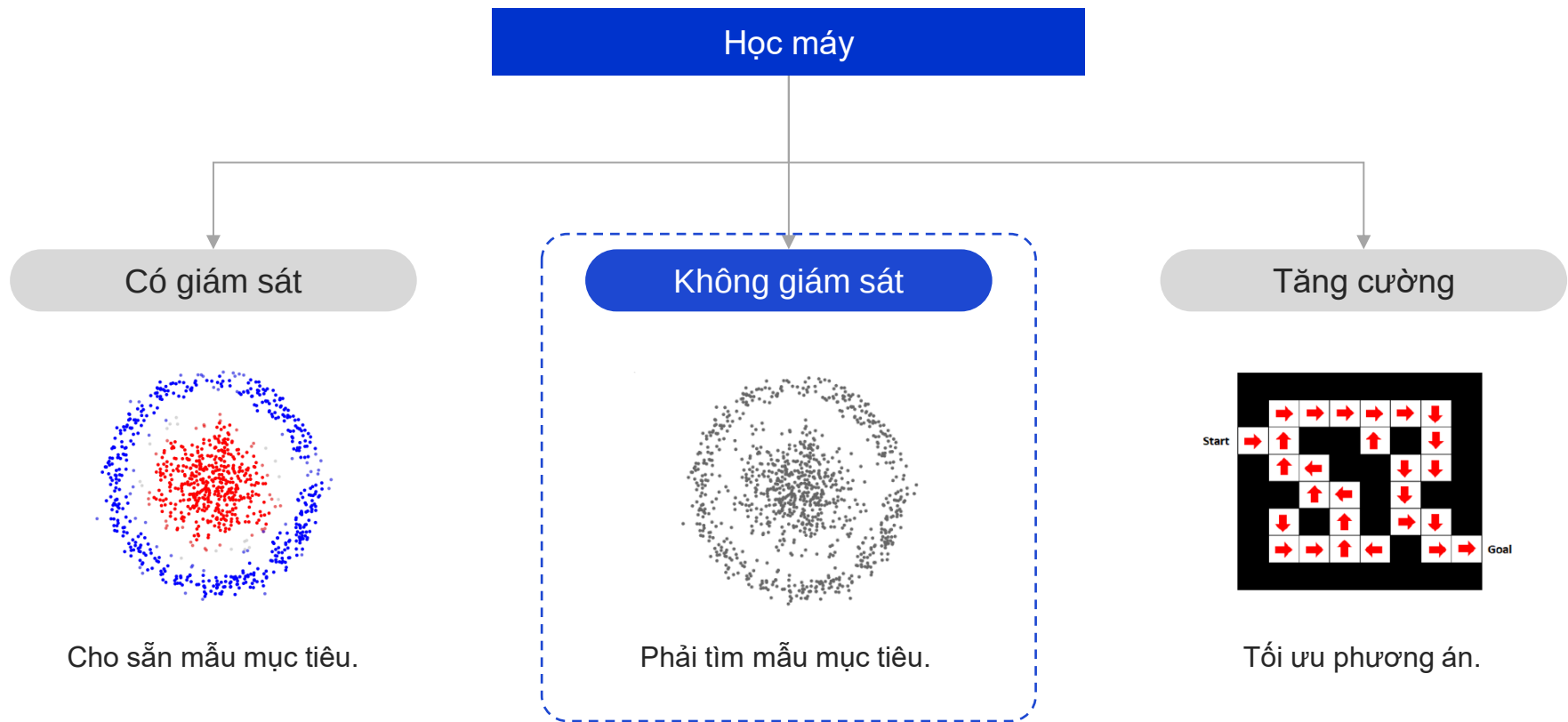
Bài 1.

Thuật toán học máy không giám sát

| 1.1. Khái niệm về học không giám sát

| 1.2. Phân tích cụm

Phân loại học máy



Học không giám sát là gì?

| Khái niệm học không giám sát

- Đây là một kỹ thuật học máy chỉ được thực hiện nếu một loạt các biến như $x_1, x_2, x_3, \dots, x_p$ được đưa ra mà không có bất kỳ biến mục tiêu nào (hoặc biến phản hồi Y) trong tập dữ liệu.
- Vì không có biến mục tiêu (hoặc biến phản hồi) liên quan đến biến giải thích, nên mục tiêu của học không giám sát là khám phá một mẫu cụ thể hoặc kiến thức chưa biết từ dữ liệu đã cho thay vì đưa ra dự đoán.

Học không giám sát là gì?

I Học máy cho học tập không giám sát

- **Phân cụm** đề cập đến việc tạo các nhóm đối tượng hoặc người tương tự nhau theo mục đích phân tích và thuật toán
- **Liên kết** đề cập đến việc khám phá mối quan hệ cùng xuất hiện giữa các mục cụ thể trong tập dữ liệu
- **Giảm chiều** đề cập đến việc tóm tắt một tập hợp biến đã cho thành một số lượng biến nhỏ hơn để đưa ra các giải thích hiệu quả.

Bài 1.

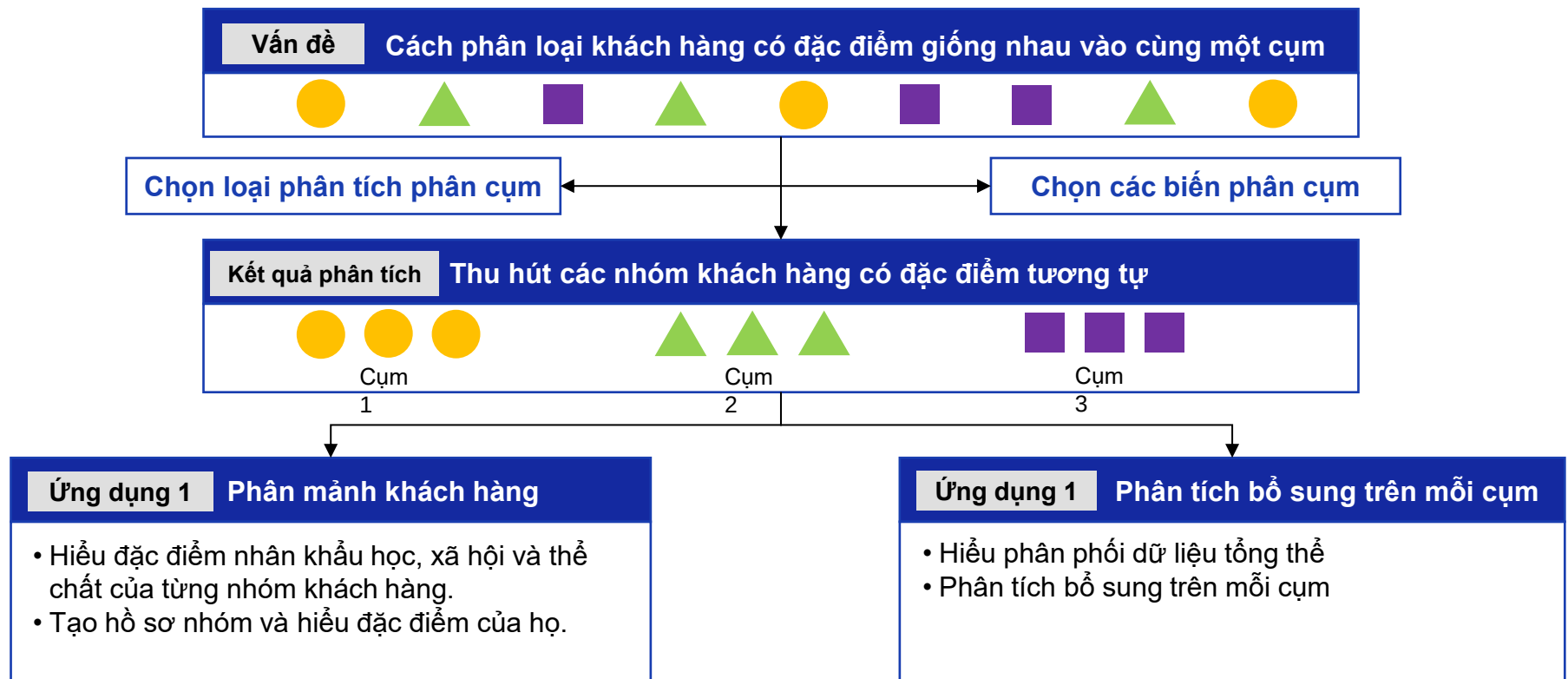
Thuật toán học máy không giám sát

| 1.1. Khái niệm về học không giám sát

| 1.2. Phân tích cụm

Phân tích phân cụm là gì?

Phân cụm là một kỹ thuật phân chia dữ liệu chưa được gắn nhãn và chưa được phân loại thành các nhóm tương tự nhau dựa trên các giá trị được quan sát đã cho.

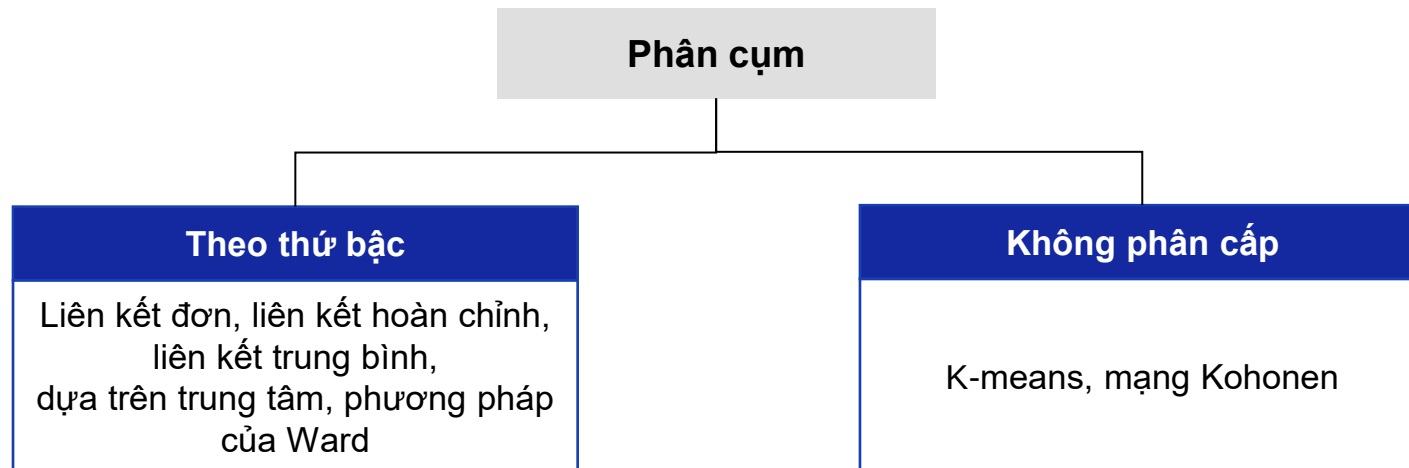


Phân tích phân cụm

- Kỹ thuật phân cụm là loại học không giám sát phổ biến nhất để nhóm dữ liệu không được gắn nhãn thành các giá trị được quan sát tương tự nhau dựa trên các tiêu chí phù hợp.
- Bởi vì phân cụm xử lý các nhóm không xác định trong dữ liệu, các nhà phân tích không thể hướng dẫn chương trình máy tính những gì cần tìm.
- Vì vậy, phân cụm chủ yếu được sử dụng để biết các thuộc tính của các nhóm khác nhau và cung cấp cho chúng ta cái nhìn sâu sắc về các mẫu cơ bản của các nhóm khác nhau hơn là để đưa ra dự đoán.

Loại phân cụm

- **Phân cụm phân cấp:** Chia rẽ và kết tụ
- **Phân cụm không phân cấp:** số lượng cụm được xác định trước



I Loại phân cụm

- Nói chung, phân tích phân cụm được phân thành Phân cụm phân cấp, Phân cụm không phân cấp hoặc phân cụm dựa trên phân vùng.

Phân cụm không phân cấp (Phân cụm dựa trên phân vùng)	Phân cụm K-Means
	Phân cụm K-Medoids hoặc PAM (Phương pháp phân vùng xung quanh)
	DBSCAN (Phân cụm ứng dụng không gian dựa trên mật độ có nhiễu)
	Bản đồ tự tổ chức
	Phân cụm mờ
Phân cụm phân cấp	Phân cụm kết hợp hoặc từ dưới lên
	Phân cụm chia nhóm hoặc từ trên xuống
Phân cụm phân phối hỗn hợp	Mô hình hỗn hợp Gaussian

Ứng dụng phân tích phân cụm

- Xác định mô hình của các nhóm dữ liệu chưa biết trong phân tích dữ liệu tài chính
- Phát hiện mọi xâm nhập mạng (các hoạt động xâm nhập trái phép vào mạng máy tính)
- Nhận dạng đối tượng và khuôn mặt từ hình ảnh kỹ thuật số
- Nhóm các tài liệu, nhạc hoặc phim thành các chủ đề khác nhau
- Phân khúc khách hàng trong lĩnh vực marketing

Bài 2.

Phân cụm phân cấp

| 2.1. Phương pháp phân cấp

Phương pháp phân cấp

Phương pháp phân cấp rất hữu ích để sắp xếp các cụm trong cấu trúc phân cấp tự nhiên. Có hai phương pháp Phân cụm phân cấp, đó là phân cụm kết hợp và phân chia.

- Phân cụm kết hợp
 - Từ dưới lên
 - Bắt đầu với n cụm để dần dần kết hợp các cụm tương tự cho đến khi còn lại một cụm cuối cùng.
- Phân cụm chia
 - Từ trên xuống
 - Bắt đầu từ một cụm duy nhất bao gồm tất cả các bản ghi và chia nó thành n cụm (phân cụm kết hợp là phương pháp phân cấp được sử dụng chủ yếu.)

| Làm thế nào để **Phân cụm kết hợp** hoạt động?

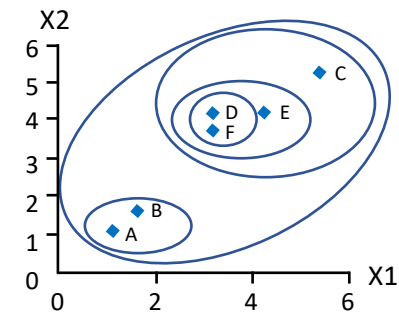
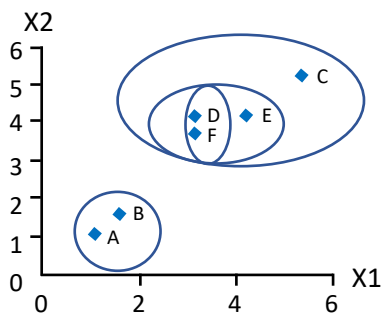
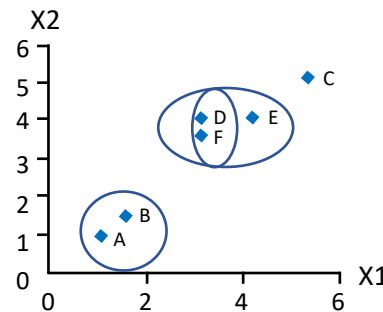
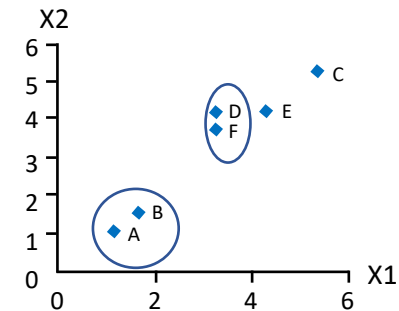
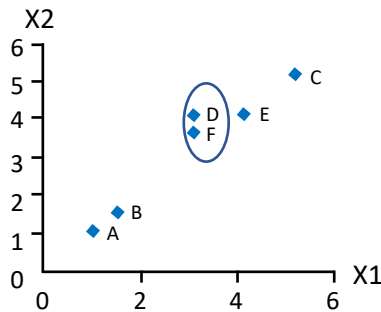
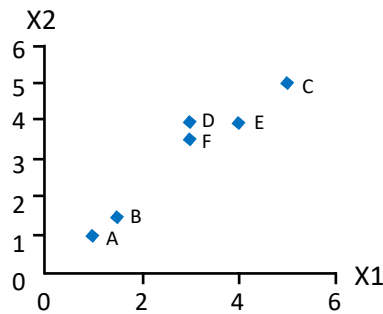
- Bắt đầu với các cụm bản ghi đơn lẻ
- Các cặp cụm tương tự được hợp nhất cho đến khi tất cả các cụm được hợp nhất thành một cụm lớn chứa tất cả các bản ghi.

| Thuật toán **Phân cụm kết hợp** là gì?

- Bắt đầu với cụm từ n
- Hợp nhất hai cụm bản ghi gần nhất thành một cụm
- Hai cụm từ gần nhất được hợp nhất trong mỗi bước. Điều này đề cập đến việc bổ sung các bản ghi đơn lẻ vào cụm hiện có hoặc kết hợp hai cụm hiện có.

Phân cụm phân cấp kết hợp

- ▶ Bắt đầu: Mỗi đối tượng là một cụm riêng lẻ.
- ▶ Sự lặp lại: Sự kết hợp lặp đi lặp lại của hai cụm gần nhất
- ▶ Kết thúc: Tạo một cụm duy nhất



Phân cụm phân cấp

| Về Phân cụm phân cấp:

- Đây là một thuật toán học không giám sát: chỉ có các biến.
- Thuật toán này còn có tên gọi là “agglomerative clustering”.
- Các mục gần nhất được gom lại để tạo thành cụm lớn hơn.
- Có thể hiển thị như một cấu trúc cây đảo ngược gọi là “dendrogram”.

| Mục đích của Phân cụm phân cấp:

- Phân cụm các quan sát thành một số lượng cụm cho trước.

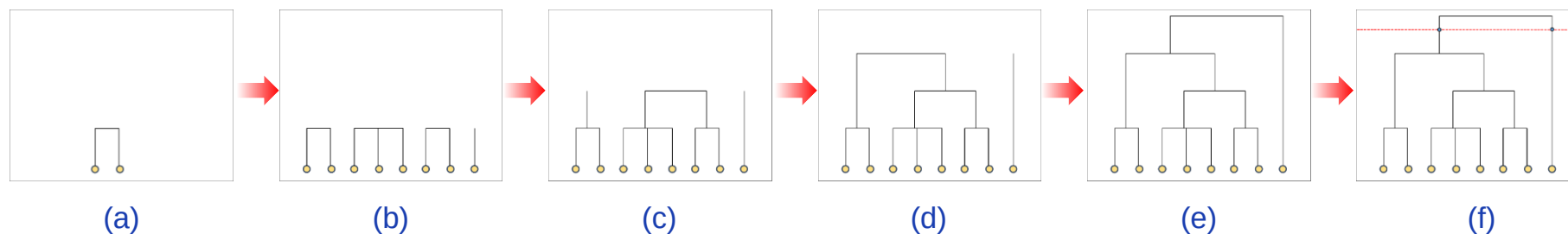
| Ưu điểm:

- Diễn giải trực quan kết quả.
- Dendrogram có thể được phân nhỏ thêm để cho ra số lượng cụm mong muốn.

| Nhược điểm:

- Tốn nhiều thời gian hơn k-means một chút.

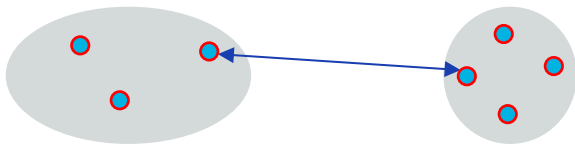
Thuật toán Phân cụm phân cấp:



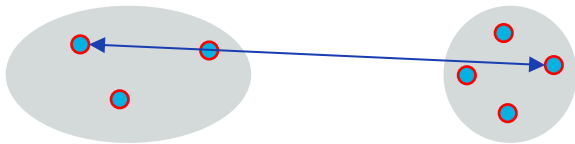
- 1) Đầu tiên, gom các quan sát gần nhất lại thành một nhóm và lấy các cụm nhỏ: (a) ~ (b).
- 2) Gom các cụm gần nhất lại thành nhóm để lập một cụm lớn hơn: (c).
- 3) Lặp lại từ bước 2) tới khi toàn bộ quan sát được gom lại thành một cụm duy nhất: (d) ~ (e).
- 4) Cắt dendrogram tại chiều cao thích hợp để có được số lượng cụm mong muốn: (f).

Thuật toán Phân cụm phân cấp:

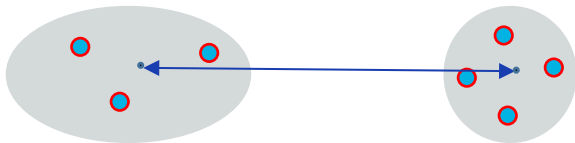
- Làm thế nào để đo khoảng cách giữa các cụm?



Liên kết đơn (Single Linkage)



Liên kết hoàn chỉnh (Complete Linkage)



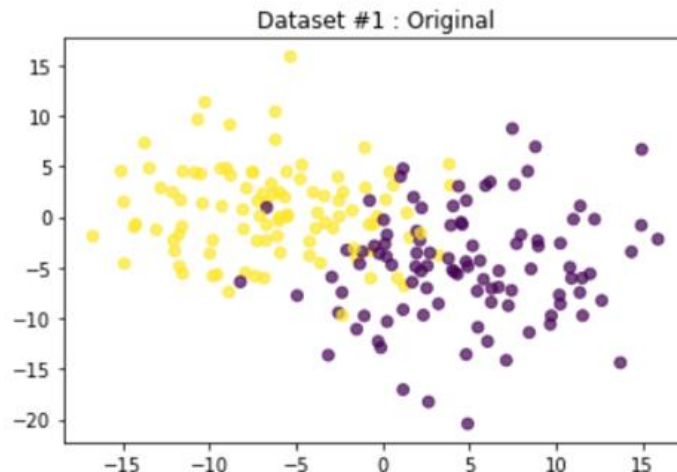
Liên kết trung bình (Average Linkage)

Áp dụng Phân cụm kết hợp và trực quan hóa:

I Tập dữ liệu số 1

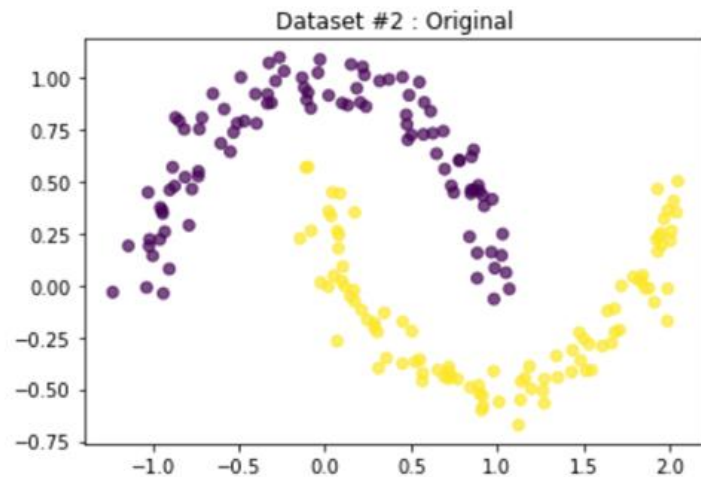
```
In [1]: from sklearn.datasets import make_blobs, make_moons  
import matplotlib.pyplot as plt
```

```
In [2]: X1, label1 = make_blobs(n_samples=200, n_features=2, centers=2, cluster_std = 5, random_state=123)  
plt.scatter(X1[:,0],X1[:,1], c= label1, alpha=0.7 )  
plt.title('Dataset #1 : Original')  
plt.show()
```



Tập dữ liệu số 2

```
In [3]: X2, label2 = make_moons(n_samples=200, noise=0.08, random_state=123)
plt.scatter(X2[:,0],X2[:,1], c= label2, alpha=0.7 )
plt.title('Dataset #2 : Original')
plt.show()
```



Các cụm

```
In [4]: from sklearn.cluster import AgglomerativeClustering
```

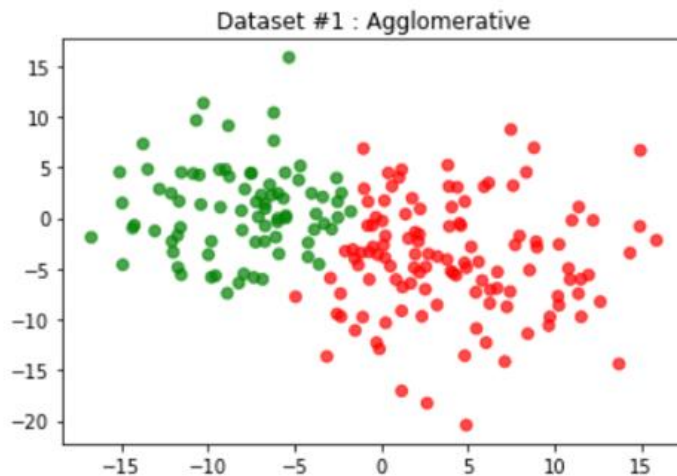
```
In [5]: import numpy as np
import pandas as pd
```

```
In [6]: X1
```

```
Out[6]: array([[ 4.83455936e+00,  1.61209639e+00],
 [-7.19342110e+00,  1.57331116e+00],
 [-1.19919003e+01, -3.38784956e+00],
 [ 8.94965320e+00, -2.34628131e+00],
 [ 5.46630481e+00, -7.33334000e+00],
 [-9.82679362e-01,  2.87439331e+00],
 [ 4.82713114e+00, -1.35870919e+01],
 [-5.95639441e+00, -4.86198432e-02],
 [ 3.25723451e+00, -3.81242660e+00],
 [-1.49388489e+01,  1.46494770e+00],
 [-9.32588959e+00,  4.75039194e+00],
 [ 1.23752259e+00, -6.81998526e+00],
 [ 1.19728386e+01, -5.62837526e+00],
 [-2.58199902e-01, -1.23070271e+01],
 [ 1.49590341e+01,  6.65671714e+00],
 [ 2.25314845e-01, -3.91267708e+00],
 [-5.93898363e+00,  2.41971297e+00],
 [-4.66024882e+00,  5.12509843e+00],
 [-2.49261965e+00,  3.07334679e-02],
 [ 7.73413068e+00, -2.65986906e+00],
 [-5.12306455e-01, -7.42476287e-01],
 [ 1.57887480e+00, -2.01725863e+00],
 [-4.20565001e+00, -2.44769569e+00],
 [-4.00859899e+00,  2.42460873e+00],
 [ 3.91603180e+00,  3.10476808e+00],
 [-1.05722363e+00, -9.77742886e+00])
```

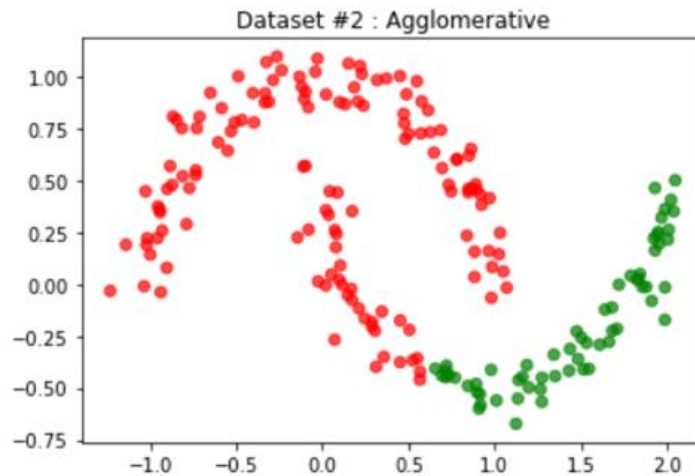

I Tập dữ liệu số 1 và hai cụm

```
In [7]: agglo = AgglomerativeClustering(n_clusters=2)
agglo.fit(X1)
myColors = {0:'red',1:'green'} # Define a color palette: 0~1.
plt.scatter(X1[:,0],X1[:,1], c= pd.Series(agglo.labels_).apply(lambda x: myColors[x]), alpha=0.7 )
plt.title('Dataset #1 : Agglomerative')
plt.show()
```



I Tập dữ liệu số 2 và hai cụm

```
In [8]: aggro = AgglomerativeClustering(n_clusters=2)
agglo.fit(X2)
myColors = {0:'red',1:'green'}
plt.scatter(X2[:,0],X2[:,1], c= pd.Series(agglo.labels_).apply(lambda x: myColors[x]), alpha=0.7 )
plt.title('Dataset #2 : Agglomerative')
plt.show()
```

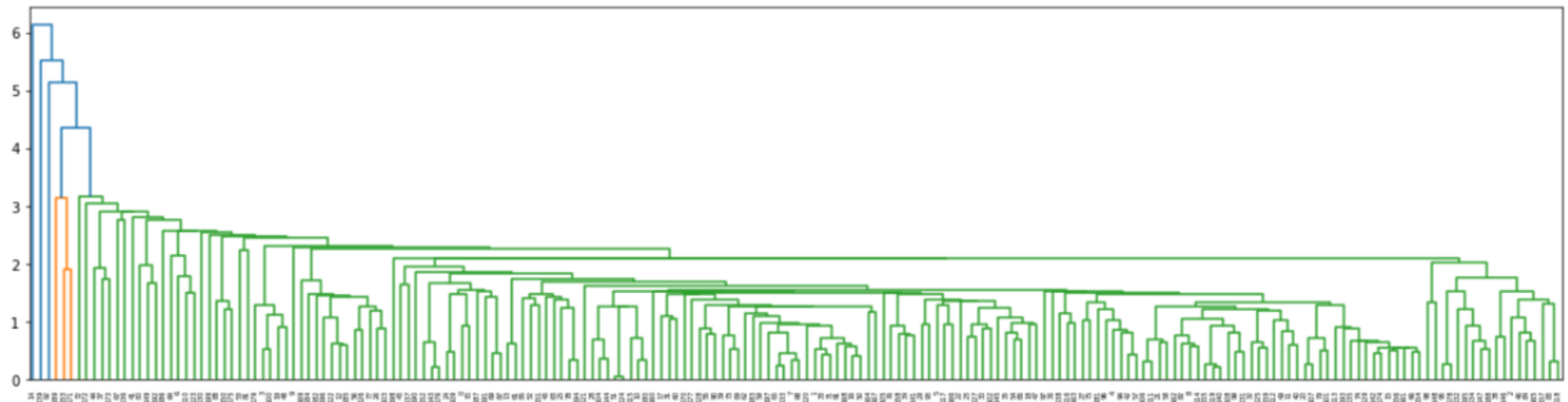


Áp dụng Phân cụm phân cấp và trực quan hóa:

- Tập dữ liệu số 1 và hiển thị dendrogram.
- Phân cụm phân cấp bằng cách sử dụng liên kết đơn

```
In [9]: from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
```

```
In [10]: myLinkage = linkage(X1,method='single')  
plt.figure(figsize=(20,5))  
dendrogram(myLinkage)  
plt.show()
```



| Áp dụng Phân cụm phân cấp và trực quan hóa:

- Bộ dữ liệu số 1 và các cụm bằng cách cắt dendrogram
- Cắt ở độ cao (khoảng cách) = 5 (thay đổi giá trị này tùy ý).

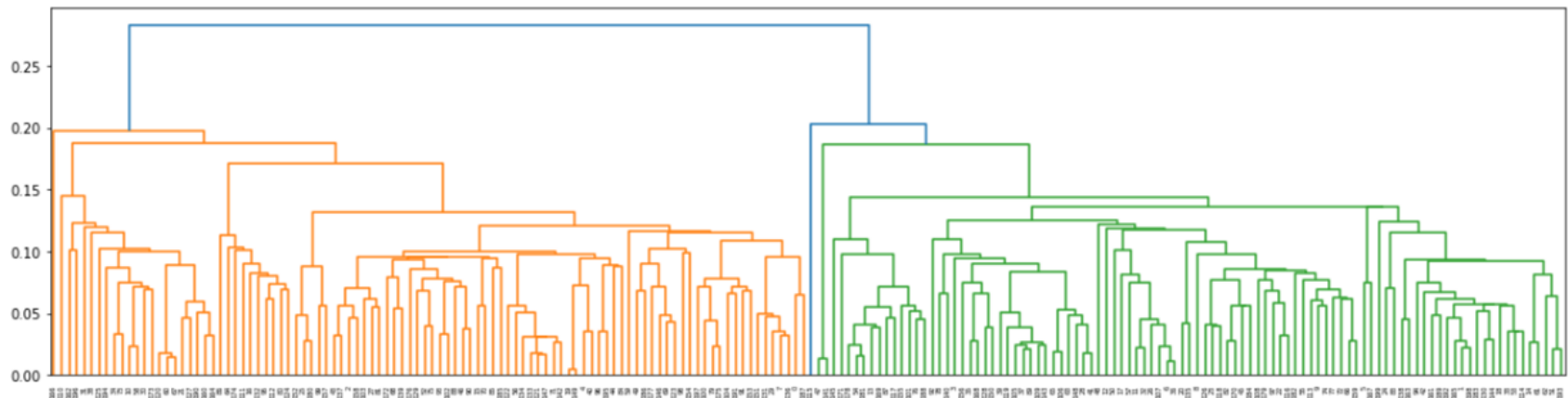
```
In [11]: labels = fcluster(myLinkage, 5, criterion='distance')  
pd.Series(labels).value_counts()
```

```
Out[11]: 1    197  
         2     1  
         3     1  
         4     1  
dtype: int64
```

Áp dụng Phân cụm phân cấp và trực quan hóa:

- Bộ dữ liệu số 2 và hiển thị dendrogram.
- Phân cụm phân cấp bằng cách sử dụng liên kết đơn.

```
In [12]: myLinkage = linkage(X2,method='single')
plt.figure(figsize=(20,5))
dendrogram(myLinkage)
plt.show()
```



| Áp dụng Phân cụm phân cấp và trực quan hóa:

- Bộ dữ liệu số 2 và các cụm bằng cách cắt dendrogram
- Cắt ở chiều cao (khoảng cách) = 0,23 (thay đổi giá trị này tùy ý).

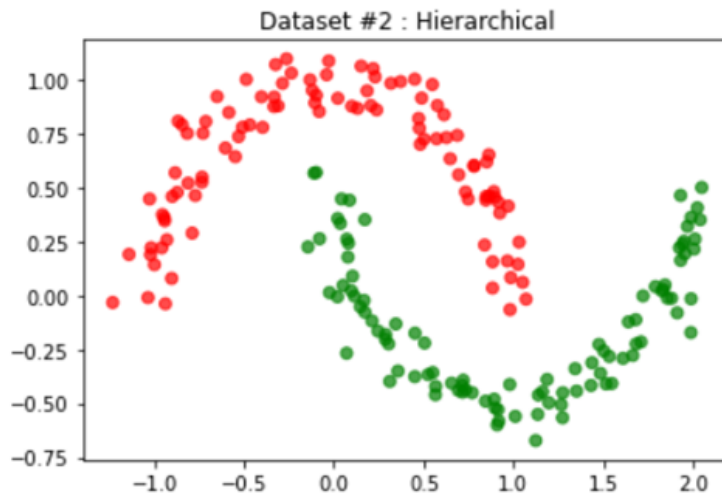
```
In [13]: labels = fcluster(myLinkage, 0.23, criterion='distance')  
pd.Series(labels).value_counts()
```

```
Out[13]: 1    100  
         2    100  
         dtype: int64
```

Áp dụng Phân cụm phân cấp và trực quan hóa:

- Xác định bảng màu: 1~2

```
In [14]: myColors = {1:'red',2:'green'}  
plt.scatter(X2[:,0],X2[:,1], c= pd.Series(labels).apply(lambda x: myColors[x]), alpha=0.7 )  
plt.title('Dataset #2 : Hierarchical')  
plt.show()
```



I Tạo Phân cụm phân cấp

- Sử dụng phương pháp kết hợp để tạo Phân cụm phân cấp.
- Nhập dữ liệu iris

```
In [1]: from sklearn.datasets import load_iris
```

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import math
%matplotlib inline
```

```
In [3]: iris=load_iris()
```


Tạo Phân cụm phân cấp

```
In [4]: iris_data=iris.data  
iris_data_pd=pd.DataFrame(iris.data, columns=iris.feature_names)  
iris_data_pd
```

Out[4]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

Tạo Phân cụm phân cấp

```
In [5]: iris_data_pd.iloc[:,2:4]
```

Out[5]:

	petal length (cm)	petal width (cm)
0	1.4	0.2
1	1.4	0.2
2	1.3	0.2
3	1.5	0.2
4	1.4	0.2
...	...	...
145	5.2	2.3
146	5.0	1.9
147	5.2	2.0
148	5.4	2.3
149	5.1	1.8

150 rows × 2 columns

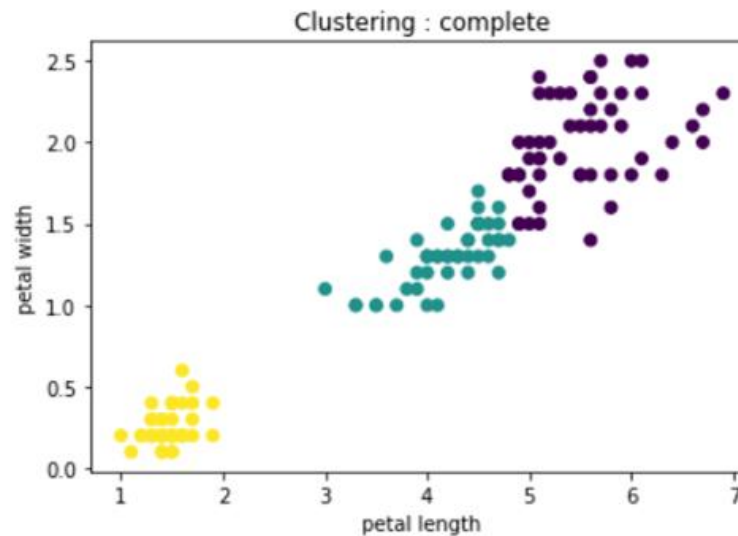
I Tạo Phân cụm phân cấp

- ▶ Để sử dụng các cột 'petal length' và 'petal width', hãy nhập Phân cụm kết hợp sklearn, được hỗ trợ bởi gói sklearn.cluster.
- ▶ Tiêu chí đo khoảng cách giữa các cụm có thể được đặt bằng tham số linkage, tham số này chỉ hỗ trợ các phương pháp ward, complete, average. Ví dụ dưới đây sử dụng cả ba tham số linkage.

```
In [6]: from sklearn.cluster import AgglomerativeClustering
        linkage=["complete", "average", "ward"]
        for idx, i in enumerate(linkage):
            plt.figure(idx)
            hier=AgglomerativeClustering(n_clusters=3, affinity='euclidean', linkage=i)
            hier.fit(iris_data_pd.iloc[:,2:4])
            plt.scatter(iris_data_pd.iloc[:,2], iris_data_pd.iloc[:,3],c=hier.labels_)
            plt.title("Clustering : " + i)
            plt.xlabel('petal length')
            plt.ylabel('petal width')
            plt.show()
```

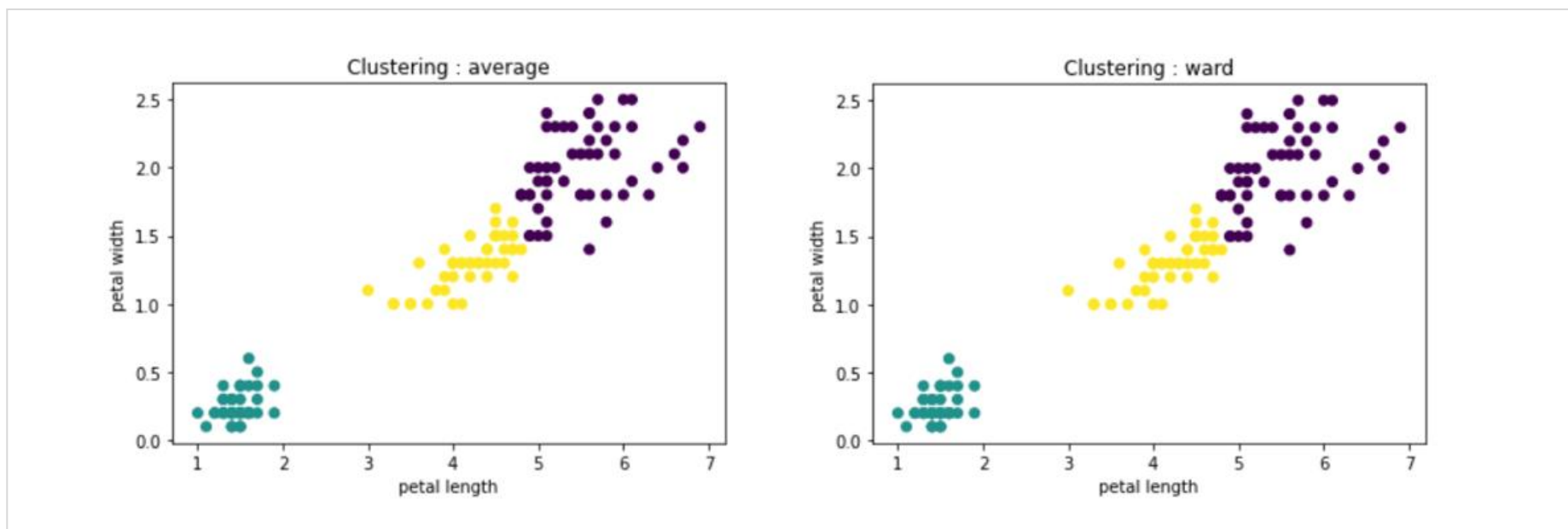
Tạo Phân cụm phân cấp

- ▶ Để sử dụng các cột 'petal length' và 'petal width', hãy nhập Phân cụm kết hợp sklearn, được hỗ trợ bởi gói sklearn.cluster.
- ▶ Tiêu chí đo khoảng cách giữa các cụm có thể được đặt bằng tham số linkage, tham số này chỉ hỗ trợ các phương pháp ward, complete, average. Ví dụ dưới đây sử dụng cả ba tham số linkage.



Tạo Phân cụm phân cấp

- ▶ Để sử dụng các cột 'petal length' và 'petal width', hãy nhập Phân cụm kết hợp sklearn, được hỗ trợ bởi gói sklearn.cluster.
- ▶ Tiêu chí đo khoảng cách giữa các cụm có thể được đặt bằng tham số linkage, tham số này chỉ hỗ trợ các phương pháp ward, complete, average. Ví dụ dưới đây sử dụng cả ba tham số linkage.



| Tạo Phân cụm phân cấp

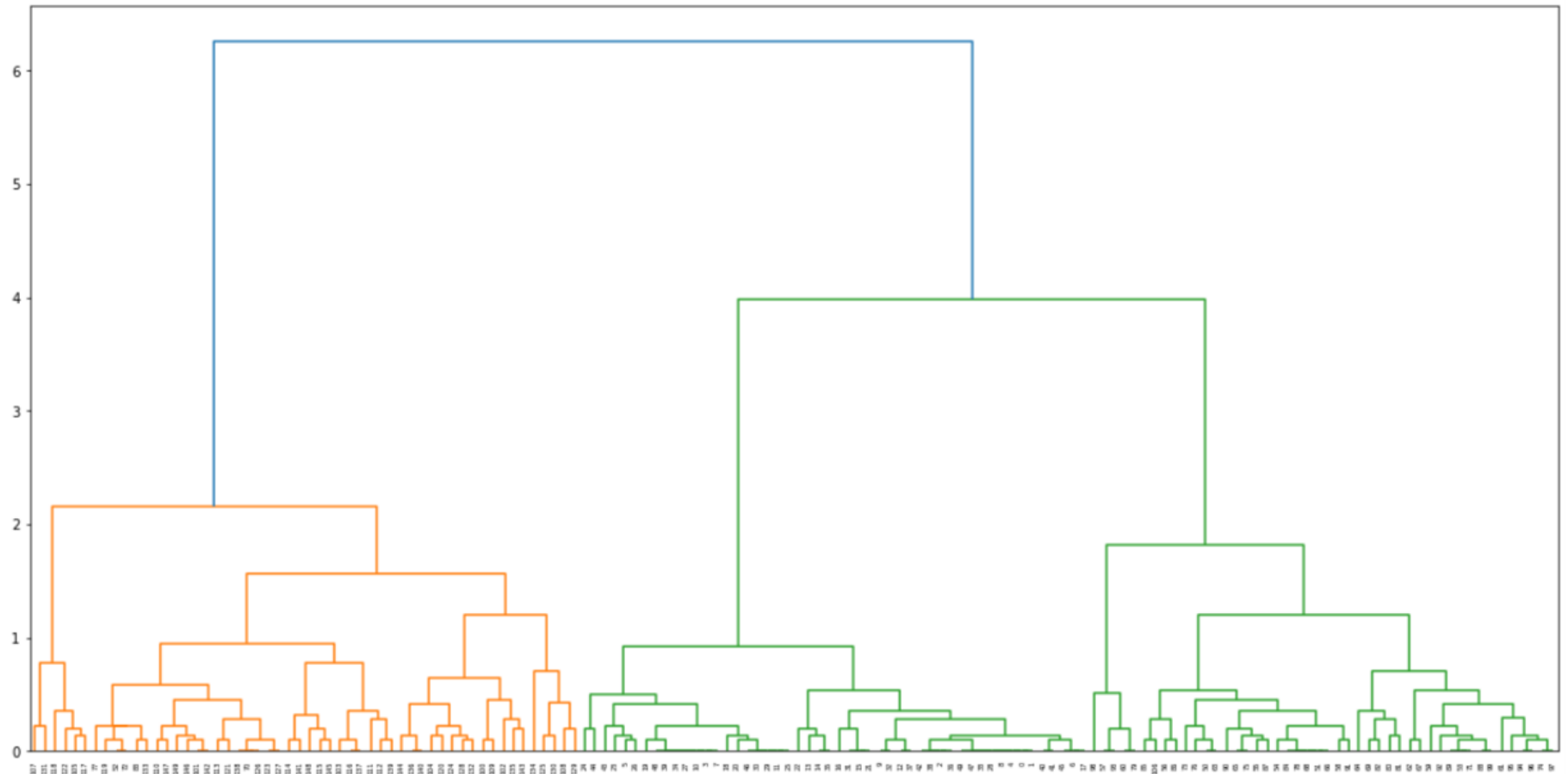
- Sử dụng Scikit-learning để xác nhận kết quả phân cụm. Gói scikit-learning không cung cấp chức năng vẽ cây.
- Vì vậy, hãy sử dụng gói scipy để phân cụm và vẽ cây.
- Ví dụ sau đây sử dụng liên kết đầy đủ

| Tạo Phân cụm phân cấp

```
In [7]: from scipy.cluster import hierarchy

In [8]: hierar=hierarchy.linkage(iris_data_pd.iloc[:,2:4], 'complete')
plt.figure(figsize=(20,10))
dn=hierarchy.dendrogram(hierar)
```

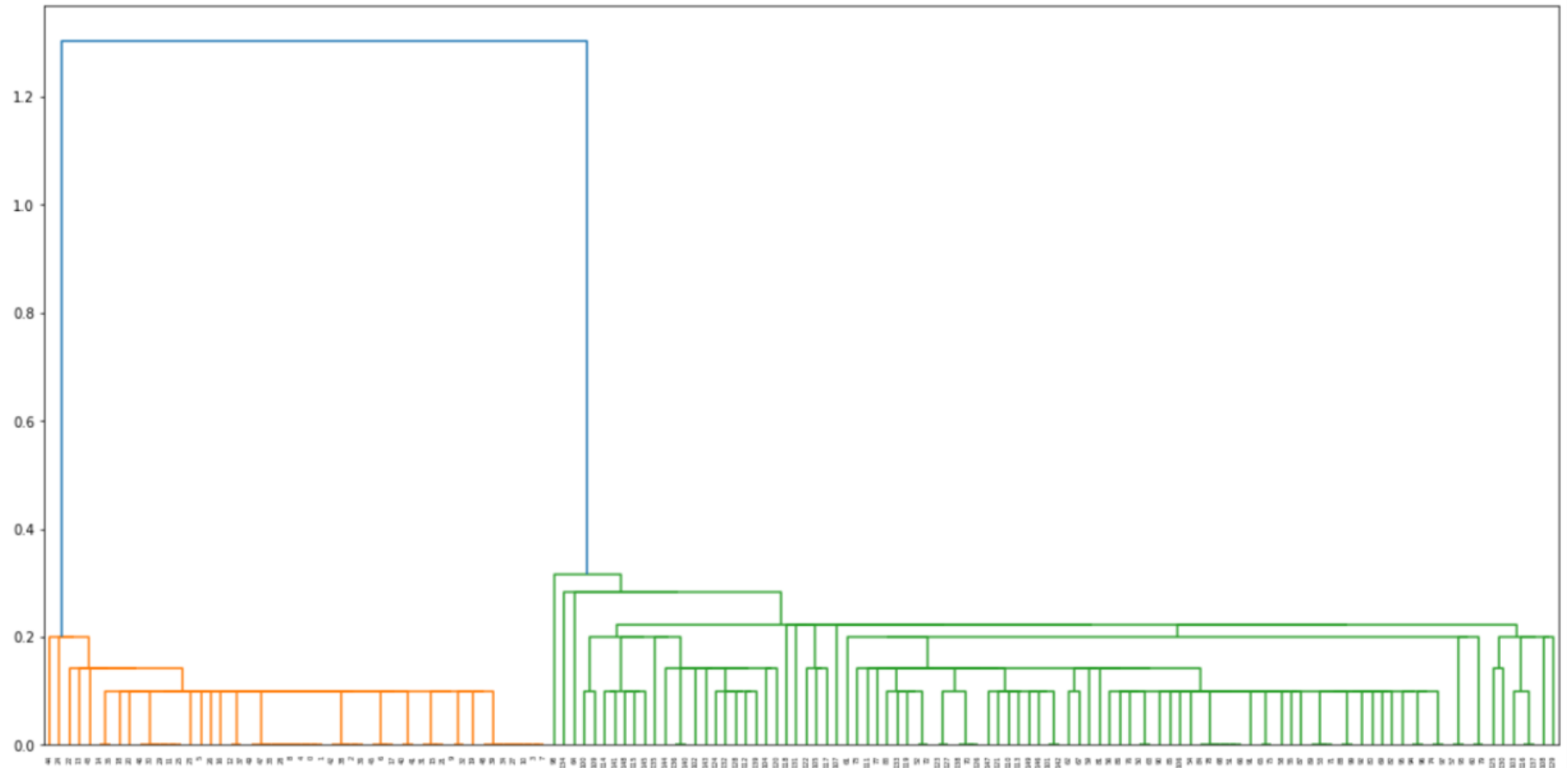
Tạo Phân cụm phân cấp



| Tạo Phân cụm phân cấp

```
In [9]: hierar=hierarchy.linkage(iris_data_pd.iloc[:,2:4], 'single')  
plt.figure(figsize=(20,10))  
dn=hierarchy.dendrogram(hierar)
```

Tạo Phân cụm phân cấp



| Tạo Phân cụm phân cấp

- Không giống như liên kết hoàn chỉnh, **liên kết đơn tạo Phân cụm phân cấp từ các phần tử gần nhất**. Nó rất khác với K-means ở chỗ liên kết đơn tạo ra một cụm phân cấp hình cây.
- Chọn một loại liên kết thích hợp để tạo ra một mô hình phù hợp với điều kiện dữ liệu là rất quan trọng.

Bài 3.

Phân cụm không phân cấp

| 3.1. Phân cụm K-means

| 3.2. Các phương pháp phân cụm khác

Phân cụm K-Means

| Về phân cụm k-Means

- Có một hoặc nhiều biến giải thích $X_1, X_2, \dots, X_d$
- Không có biến phản hồi. Do đó, đây là một thuật toán học tập không giám sát.

| Mục đích của phân cụm k-means

- Nhóm các quan sát thành k cụm.
- Tìm trọng tâm (trung bình của cụm) đại diện cho cụm.

| Ưu điểm:

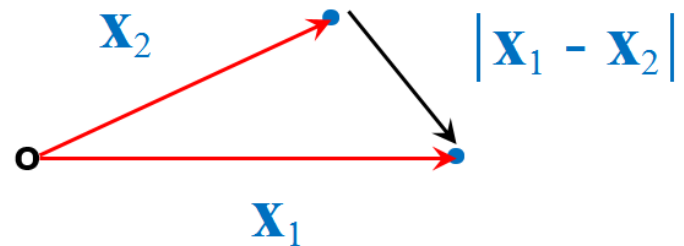
- Giải thích trực quan các kết quả.
- Nhanh chóng và dễ dàng

| Nhược điểm:

- Nhạy cảm với nhiễu và thành phần ngoại lai.
- Ranh giới cụm chỉ mang tính tuyến tính.

Chỉ số khoảng cách:

- Khoảng cách Euclid: đây là mô-đun của hiệu số giữa hai vector vị trí.



$$|X_1 - X_2| = \sqrt{(x_{11} - x_{21})^2 + (x_{12} - x_{22})^2 + \dots + (x_{1d} - x_{2d})^2}$$

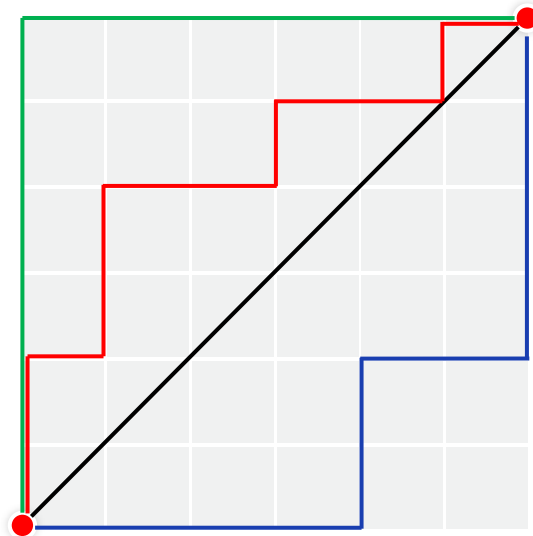
Chỉ số khoảng cách:

- Với biến số:

Euclid, Standardized, Mahalanobis, Chebyshev, Canberra, Manhattan, Minkowski, v.v.

- Với biến phân loại:

Jackard, v.v.



Đường đen: Euclidean / Đường màu: Manhattan.

Thuật toán gốc:

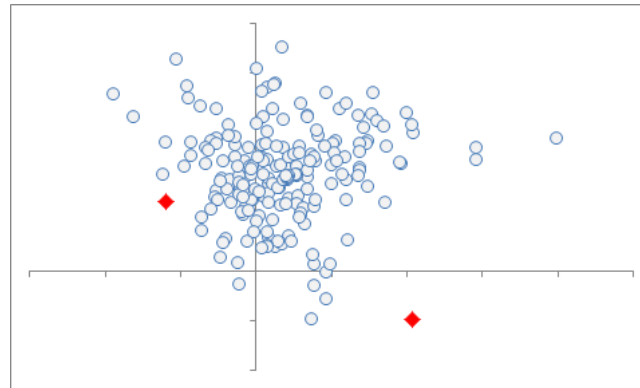
1) Giả sử có một tập dữ liệu bao gồm các quan sát:

$$x_1, x_2, \dots, x_n$$

▸ Mỗi quan sát x_i là một vector gồm d thành phần (d = số biến)

2) Giả sử rằng chúng ta nhắm mục tiêu hai cụm ($k=2$), được ký hiệu là $C1$ và $C2$.

3) Hai vị trí trọng tâm được khởi tạo ngẫu nhiên: μ_1 và μ_2 .



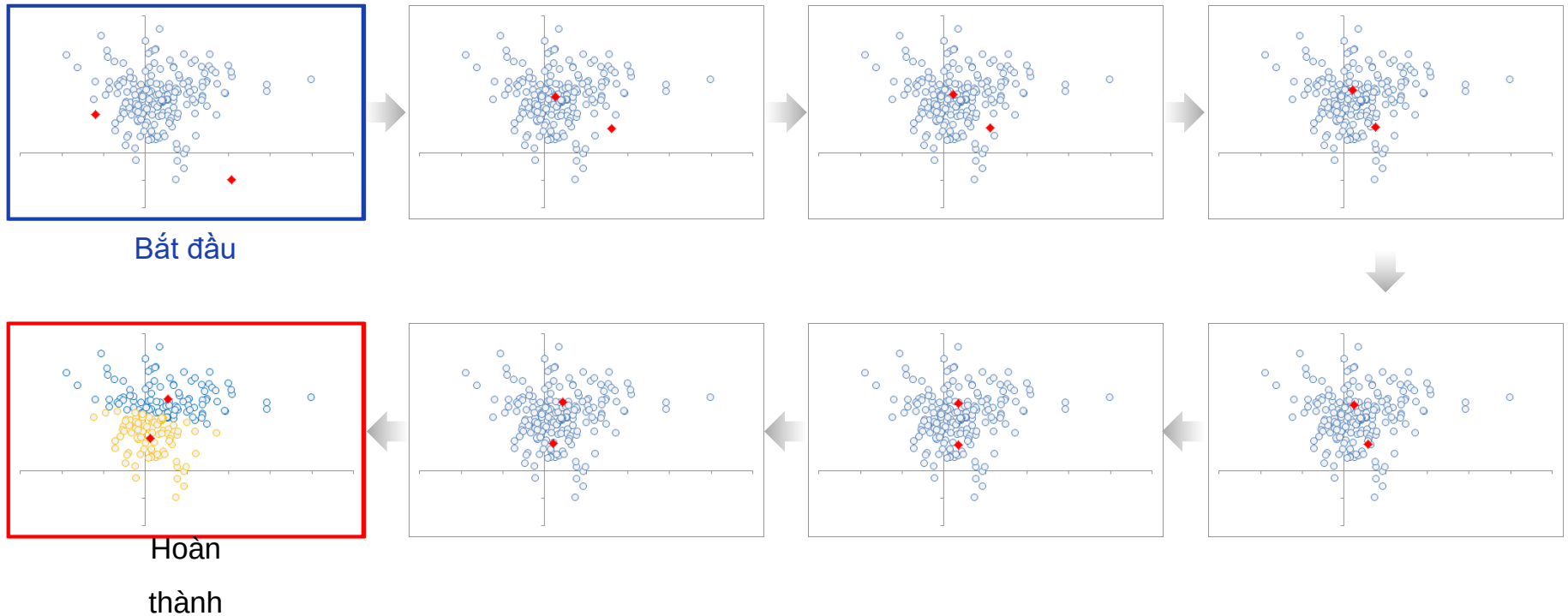
| Thuật toán gốc:

- 4) Gán các quan sát vào tâm gần nhất.
- 5) Cập nhật vị trí tâm sao cho tổng bình phương khoảng cách giữa các quan sát và tâm gần nhất là nhỏ nhất.

$$\text{minimize } \sum_{i=1}^k \sum_{x \in C_i} |x - \mu_i|^2$$

- 6) Lặp lại từ bước 4) cho đến khi các trọng tâm hội tụ về vị trí cuối cùng của chúng.

Thuật toán gốc:

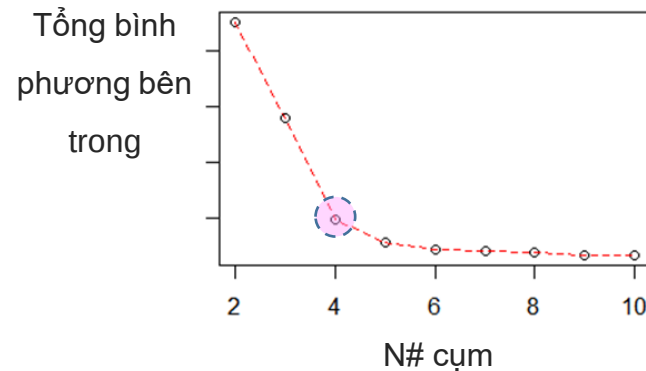


Thuật toán gốc:

- ▶ Câu hỏi: làm thế nào để tìm được số lượng cụm tối ưu?
- ▶ Xác định tổng bình phương khoảng cách giữa các quan sát và tâm gần nhất như sau:

$$\text{Tổng bình phương bên trong} = \sum_{i=1}^k \sum_{x \in C_i} |x - \mu_i|^2$$

- ▶ Vẽ đồ thị “Tổng bình phương bên trong” so với “N# cụm” và tìm điểm uốn ← “Elbow”.



Bài tập coding số 0401



Làm theo các bước thực hành trong tệp 'ex_0401.ipynb'

Bài tập coding số 0402



Làm theo các bước thực hành trong tệp 'ex_0402.ipynb'

Bài 3.

Phân cụm không phân cấp

| 3.1. Phân cụm K-means

| 3.2. Các phương pháp phân cụm khác

Các phương pháp phân cụm khác

Phân cụm K-Medoids

- ▶ Đây là phiên bản nâng cao của phương pháp phân cụm K-means và nó sử dụng tất cả các loại thước đo độ tương tự và không tương tự. Phân cụm K-Medoids thuận lợi cho xử lý nhiễu hoặc ngoại lệ vì nó chỉ định trung tâm cụm bằng cách sử dụng các giá trị tập dữ liệu thực thay vì các dấu chấm ngẫu nhiên trên tọa độ mặt phẳng.

DBSCAN (Phân cụm ứng dụng không gian dựa trên mật độ có nhiễu)

- ▶ Trong khi phương pháp phân cụm K-means tính toán giá trị trung bình của K-cụm và khoảng cách giữa mỗi điểm dữ liệu để phân cụm, thì DBSCAN áp dụng mật độ để tạo cùng một nhóm tập dữ liệu được liên kết với mật độ không đổi. Đó là một phương pháp phân cụm thuận lợi để xác định nhiễu và ngoại lệ.

Mô hình hỗn hợp Gaussian

- ▶ Phân tích phân cụm dựa trên xác suất dự đoán tham số bằng cách sử dụng thuật toán EM (Tối đa hóa kỳ vọng) hoặc MCMC (Chuỗi Markov Monte Carlo).

Phân cụm không gian dựa trên mật độ của các ứng dụng có nhiễu (DBSCAN)

I Về DBSCAN:

- Thuật toán phổ biến nhất trong phân cụm mật độ là Thuật toán phân cụm ứng dụng không gian dựa trên mật độ có nhiễu (DBSCAN). Tương tự như phân cụm K-means, phân cụm DBSCAN bị ảnh hưởng bởi phân phối dữ liệu.
- Trong khi phân cụm K-means tạo các cụm theo phân phối bằng cách sử dụng dữ liệu tiêu chuẩn, phân cụm DBSCAN bị ảnh hưởng bởi mật độ của từng dữ liệu. Nói cách khác, nó giả định rằng dữ liệu được đưa vào cùng một cụm sẽ có mật độ cao.
- Chúng ta sẽ xem cách DBSCAN xác định mật độ.

| Về DBSCAN:

- Đây là một thuật toán học không giám sát: chỉ có các biến X .
- Được phát triển vào năm 1996.
- Cho phép phân cụm dựa trên mật độ.

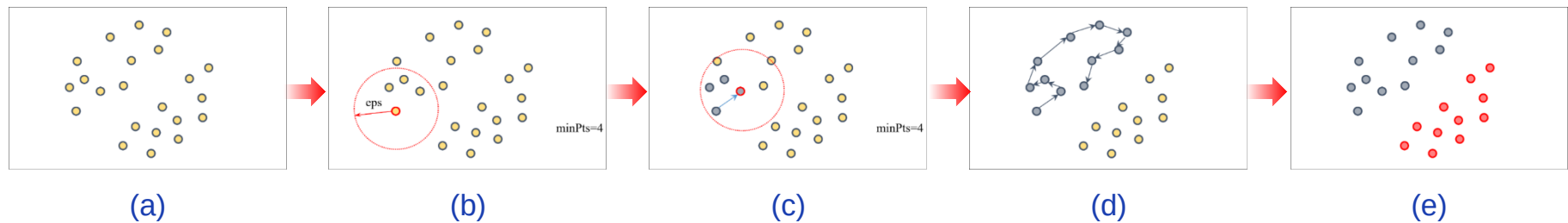
| Ưu điểm:

- Có thể đưa ra những cấu trúc mà phân cụm k-means và Phân cụm phân cấp không làm được.

| Nhược điểm:

- Phải xác định siêu tham số ϵ và minPts .
- Khó lấy được cụm ổn định nếu các vùng dày đặc bị chồng chéo nhau.

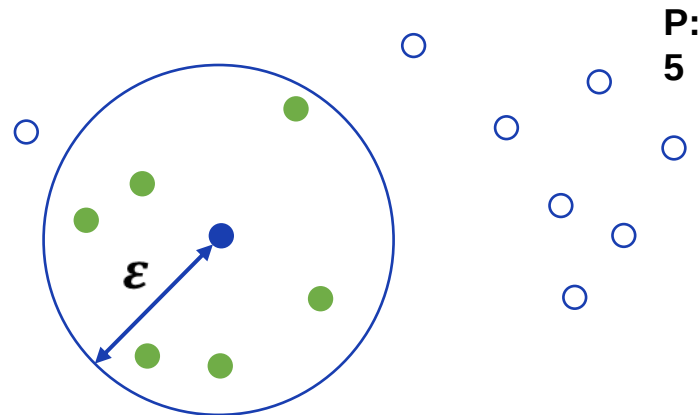
Thuật toán DBSCAN:



- 1) Giả sử các quan sát được phân bố như hình (a).
- 2) Từ một điểm, đếm các điểm nằm trong bán kính ϵ : (b).
- 3) Nếu có nhiều hơn $\minPts$ điểm nằm trong hình tròn, gom chúng vào cùng một cụm: (b).
- 4) Sau đó chuyển sang điểm tiếp theo và lặp lại từ bước 2) tới khi gom hết các điểm còn lại: (c)~(d).
- 5) Chuyển tới một điểm khác chưa được phân cụm. Lặp lại từ bước 2): (e).

■ Cách xác định mật độ cho thuật toán DBSCAN

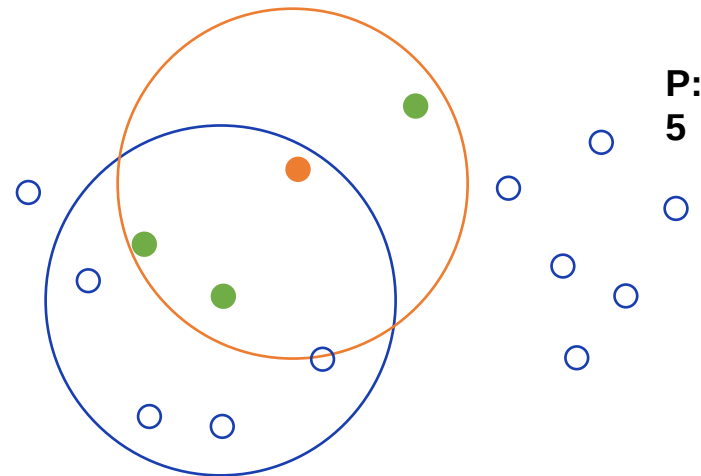
- ▶ Kiểm tra các tham số sau để xác định mật độ cho thuật toán DBSCAN.
 1. ϵ là bán kính của đường tròn có phần tử ở tâm
 2. P là số lượng phần tử tối thiểu tồn tại trong bán kính. Khoảng cách bán kính được đo bằng phương pháp khoảng cách Euclidian.
- ▶ Nhìn vào hình sau. Đầu tiên chọn một điểm ngẫu nhiên.



| Cách xác định mật độ cho thuật toán DBSCAN

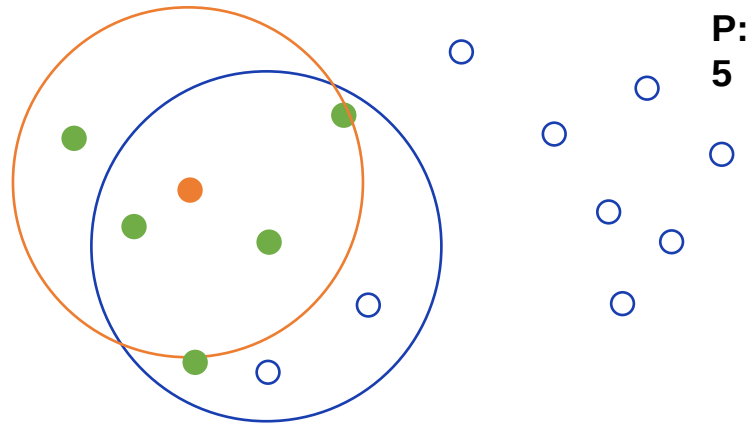
- Điểm được chọn ngẫu nhiên được chỉ định tại một dấu chấm màu xanh dương như thể hiện trong hình ở slide trước.
- Hình tròn có tâm là chấm xanh dương và có bán kính là ϵ . Kiểm tra xem số phần tử bên trong hình tròn có bằng P hoặc lớn hơn P không.
- Nếu nó bằng hoặc lớn hơn P , thì chấm xanh dương sẽ trở thành đối tượng lõi và vòng tròn được tạo thành một cụm. Nếu giá trị nhỏ hơn P , thì chấm xanh dương được xác định là nhiễu.
- Trong hình, các phần tử trong vòng tròn màu xanh dương là sáu chấm màu xanh lá cây. P bằng 5, do đó, số lượng phần tử tối thiểu tồn tại trong bán kính được thỏa mãn.
- Do đó, chấm xanh dương trở thành đối tượng lõi và vòng tròn màu xanh dương trở thành cụm. Bây giờ, chuyển sang các phần tử khác. Chọn một trong các phần tử trong vòng tròn ngoài đối tượng cốt lõi.
- Hãy nhìn vào dấu chấm mới trong hình tiếp theo.

| Cách xác định mật độ cho thuật toán DBSCAN



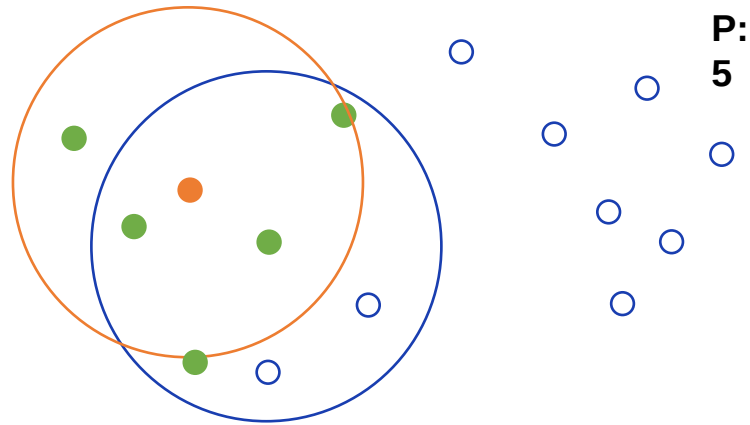
- ▶ Vòng tròn màu xanh lam là cụm hiện có và một dấu chấm mới được chỉ định trong bán kính. Khi tạo một vòng tròn có chấm cam ở tâm, chỉ có ba phần tử bên trong nó.
- ▶ Giá trị P là 5, vì vậy số lượng phần tử bên trong vòng tròn ít hơn P. Vì vậy, dấu chấm này không thể trở thành đối tượng lõi. Thay vào đó, nó được xác định là nhiễu.
- ▶ Di chuyển lên một dấu chấm khác trong bán kính.

| Cách xác định mật độ cho thuật toán DBSCAN



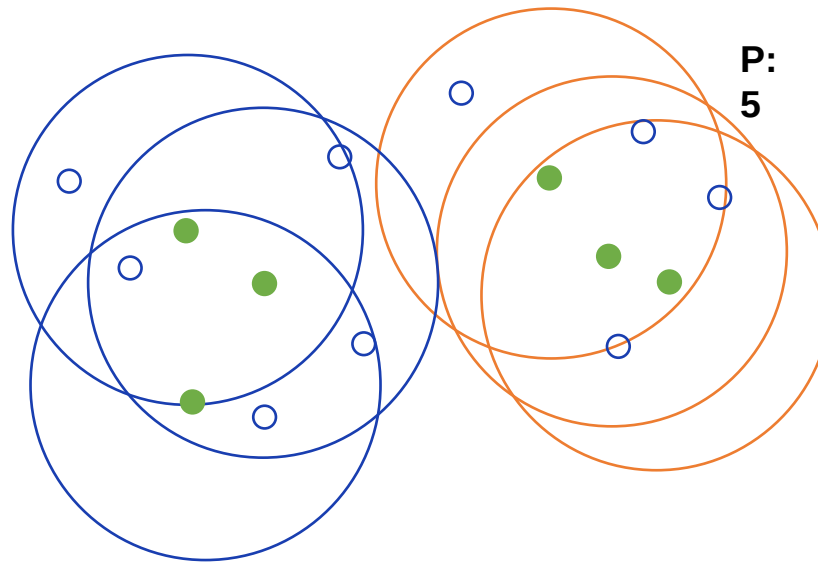
- ▶ Bây giờ, có 5 phần tử trong vòng tròn màu cam. Nó giống như giá trị P, vì vậy chấm cam này được nhận dạng là đối tượng lõi.
- ▶ Có thể tạo một cụm và đảm bảo xem xét liệu phần tử này có được bao gồm trong cụm trước đó hay không vì hiện tại nó được phân loại là một đối tượng lõi mới.
- ▶ Nếu được bao gồm trong cụm trước, không tạo cụm mới mà mở rộng cụm hiện có. Nếu nó không liên quan đến cụm trước đó, thì một cụm mới sẽ được tạo bằng cách sử dụng chấm cam làm đối tượng cốt lõi mới.
- ▶ Trong hình, chấm cam là thành phần có mật độ trong vòng tròn xanh là cụm trước đó. Bởi vì nó được bao gồm trong cụm đã tạo trước đó, nên cụm trước được mở rộng mà không cần tạo cụm mới.

| Cách xác định mật độ cho thuật toán DBSCAN



- Đây là cụm mở rộng.
- Mục đích của thuật toán DBSCAN là kiểm tra mọi dấu chấm bằng cách lặp đi lặp lại cùng một quy trình. Sau khi hoàn thành tất cả các quy trình, cụm dữ liệu trông như sau.

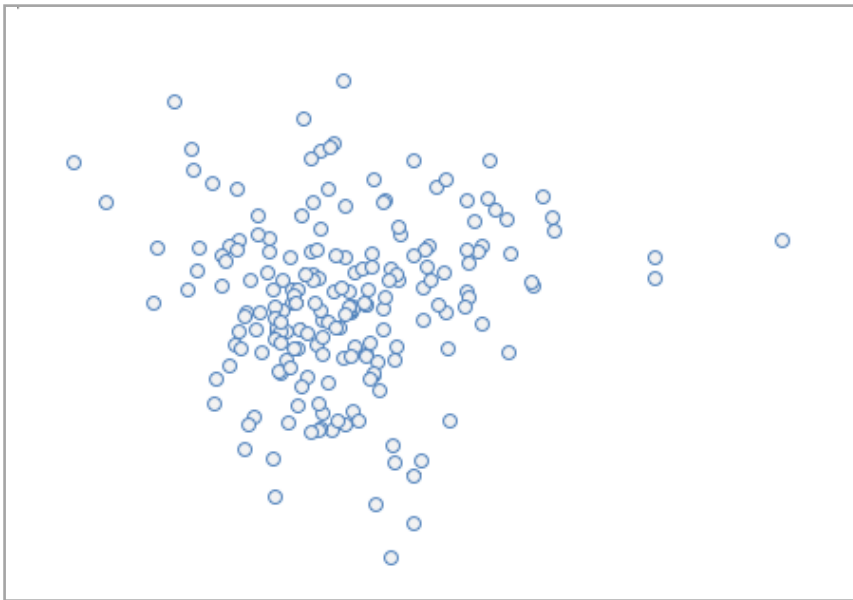
| Cách xác định mật độ cho thuật toán DBSCAN



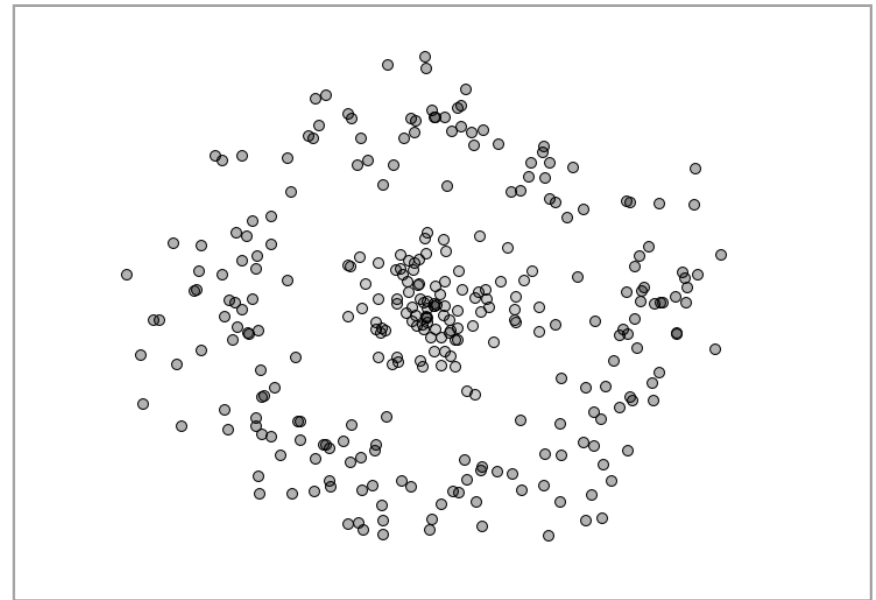
- Các chấm đen là các đối tượng lõi.
- Tương tự như vậy, DBSCAN chỉ lấy hai tham số và sử dụng chúng để tạo một cụm.
- Do đó, không có tham số nào khác được yêu cầu ngoài giá trị ϵ và P. Đây là cách mật độ được xác định trong thuật toán DBSCAN.

Phân cụm

■ Câu hỏi: Thuật toán phân cụm nào phù hợp với những trường hợp sau?



(a)



(b)

Bài tập coding số 0403



Làm theo các bước thực hành trong tệp 'ex_0403.ipynb'.

Bài 4.

Mô hình nhân tố tuyến tính để giảm số chiều dữ liệu

- | 4.1. Phân tích thành phần chính
- | 4.2. Ứng dụng của các thành phần chính

Phân tích thành phần chính

| Mục đích của phân tích thành phần chính (PCA):

- Biến đổi một tập hợp các biến tương quan thành một tập hợp các biến không tương quan khác.
- Tạo ra một bộ vectơ trực giao mới (các thành phần chính hoặc PC).
- Sắp xếp thứ tự các biến mới từ phương sai lớn nhất đến nhỏ nhất.

| Ưu điểm:

- Nhiều ứng dụng khoa học dữ liệu: tiền xử lý, mô hình hóa, giảm kích thước, trực quan hóa, v.v.

| Nhược điểm:

- Khó diễn giải vì PC có được nhờ sự kết hợp tuyến tính của các đặc trưng gốc.

| Thuật ngữ:

a) Tải:

- Thành phần chính được chuẩn hóa (PC).
- Có nhiều vector tương ứng với số lượng biến.

b) Phương sai σ^2 (hoặc độ lệch chuẩn σ):

- Các thành phần chính có phương sai liên quan.

c) Điểm quy đổi:

- Điểm thô (quan sát ban đầu) được biểu thị bằng các thành phần chính (PCs) như là các trục tọa độ mới.

Thành phần chính:

- ▶ Giả sử rằng có k biến hoặc đặc trưng $X_1, X_2, \dots, X_k$.
- ▶ Khi đó, $PC_1, PC_2, \dots, PC_k$ là tổ hợp tuyến tính của các đặc trưng gốc:

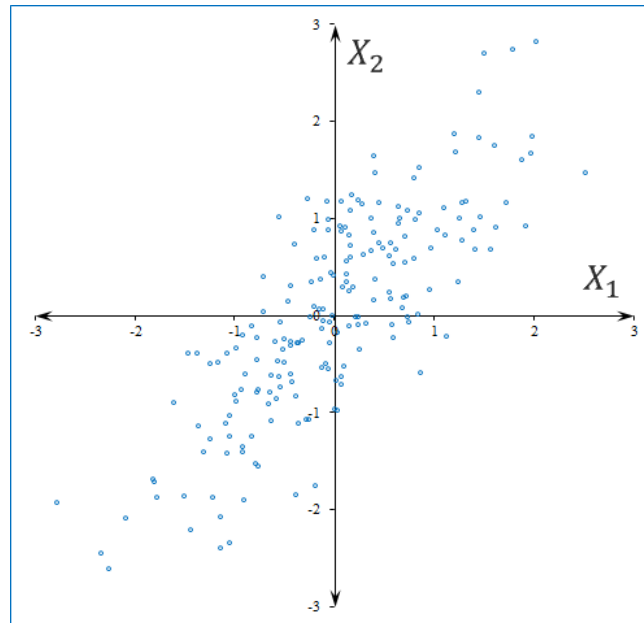
$$PC_i = \alpha_{1,i}X_1 + \alpha_{2,i}X_2 + \dots + \alpha_{k,i}X_k$$

- ▶ Ngược lại, các đặc trưng ban đầu có thể được thể hiện dưới dạng PC:

$$X_i = \beta_{1,i}PC_1 + \beta_{2,i}PC_2 + \dots + \beta_{k,i}PC_k$$

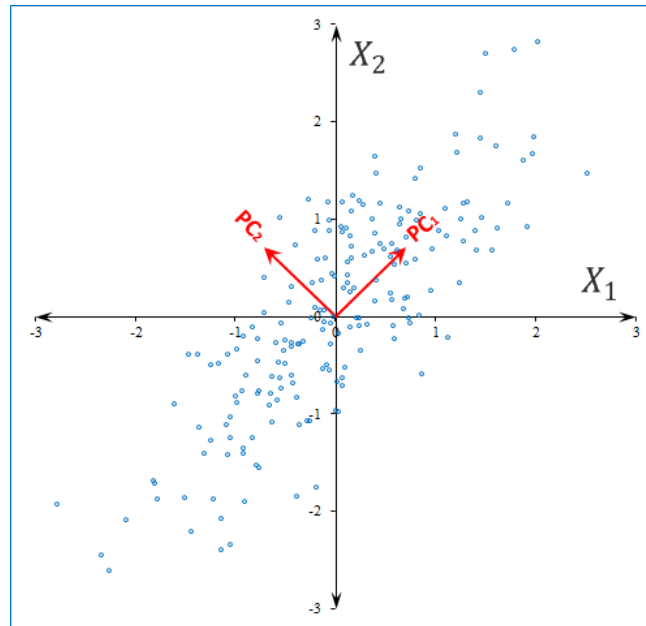
Thành phần chính và phương sai:

- Giả sử một tập dữ liệu có hai biến có thể được hiển thị trên mặt phẳng:



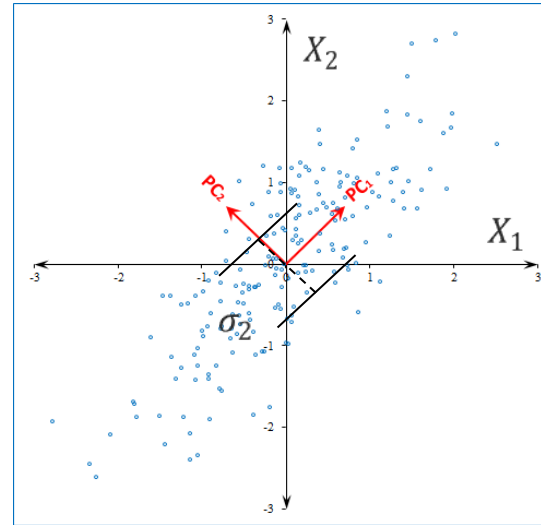
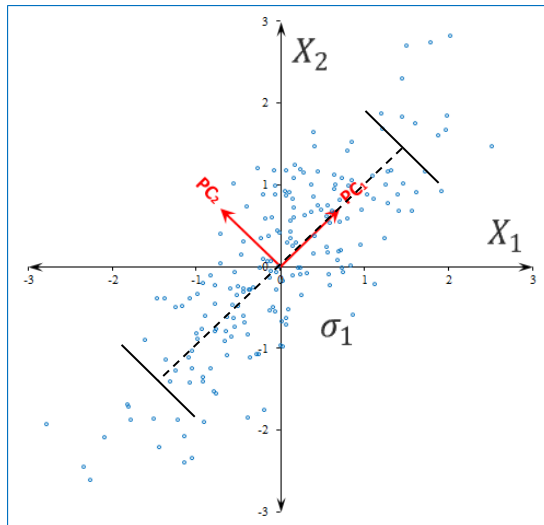
Thành phần chính và phương sai:

- Ta có thể tìm được $PC1$ và $PC2$ trực giao với nhau.



Thành phần chính và phương sai:

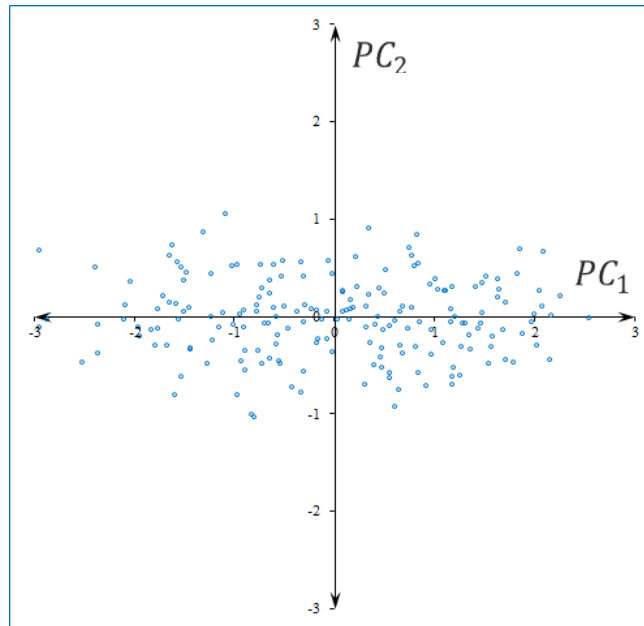
- ▶ $PC1$ là hướng của phương sai lớn nhất, trong khi $PC2$ là hướng của phương sai lớn nhất tiếp theo.



- ▶ Vậy, ta có $\sigma_1 > \sigma_2$.

■ Điểm số đã chuyển đổi:

- Các quan sát có thể được biểu diễn bằng cách sử dụng PC_1 và PC_2 làm trục tọa độ mới.



Phương sai tích lũy:

- Vì các thành phần chính có thể được coi là các biến độc lập nên tổng phương sai là:

$$\sigma_{total}^2 = \sigma_1^2 + \sigma_2^2 + \sigma_3^2 + \dots$$

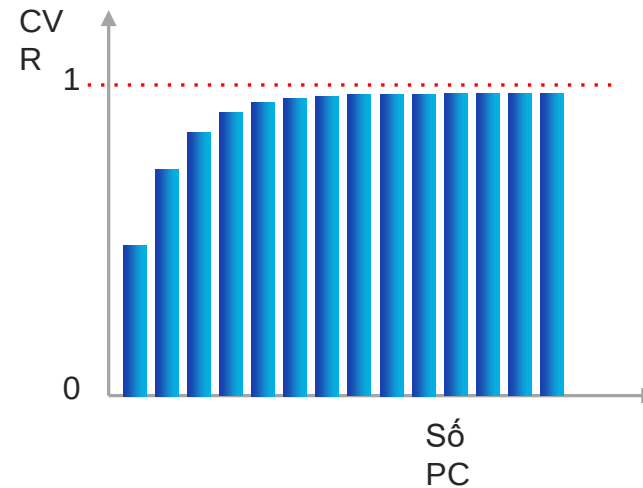
- Vì vậy, chúng ta có thể tính tỷ lệ phương sai tích lũy (CVR):

$$CVR_1 = \frac{\sigma_1^2}{\sigma_{total}^2}$$

$$CVR_2 = \frac{\sigma_1^2 + \sigma_2^2}{\sigma_{total}^2}$$

$$CVR_3 = \frac{\sigma_1^2 + \sigma_2^2 + \sigma_3^2}{\sigma_{total}^2}$$

⋮



| Tính toán các thành phần chính:

- Các PC cũng có thể thu được bằng cách phân tách giá trị riêng (ED) của ma trận hiệp phương sai.
- Các PC có thể được tính toán bằng cách phân tách giá trị số ít (SVD) của ma trận dữ liệu.
- Nếu chuẩn hóa các biến, thì thay vì hiệp phương sai, sẽ có mối tương quan.

VD

Ma trận hiệp phương sai và ma trận tương quan.

$$\begin{bmatrix} \sigma_1^2 & \sigma_{12} & \sigma_{13} & \sigma_{14} \\ \sigma_{21} & \sigma_2^2 & \sigma_{23} & \sigma_{24} \\ \sigma_{31} & \sigma_{32} & \sigma_3^2 & \sigma_{34} \\ \sigma_{41} & \sigma_{42} & \sigma_{43} & \sigma_4^2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & \rho_{12} & \rho_{13} & \rho_{14} \\ \rho_{21} & 1 & \rho_{23} & \rho_{24} \\ \rho_{31} & \rho_{32} & 1 & \rho_{34} \\ \rho_{41} & \rho_{42} & \rho_{43} & 1 \end{bmatrix}$$

“

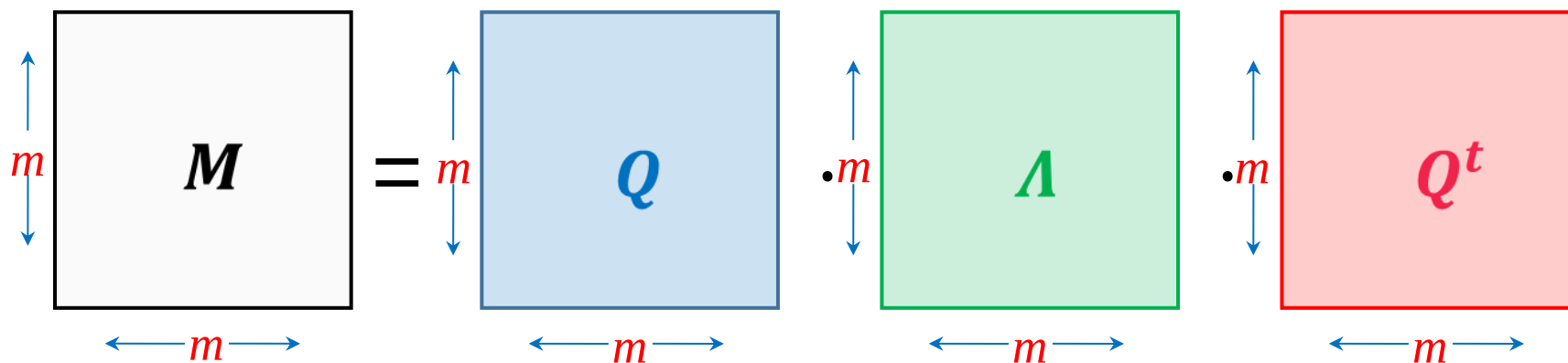
Trong một vài slide tiếp theo, chúng ta sẽ xem xét chi tiết ED và SVD

”

Phân rã ma trận

■ Phân tích giá trị riêng (ED):

- ▶ Một ma trận **vuông** M được phân rã thành $M = Q \Lambda Q^t$.
- ▶ Toàn bộ ma trận đều có cùng kích cỡ: $\text{Size}(M) = \text{Size}(Q) = \text{Size}(\Lambda) = m \times m$.



Phân tích giá trị riêng (ED):

- ▶ Một ma trận **vuông** M được phân rã thành $M=Q \Lambda Q^t$.
- ▶ Ở đây, Λ là ma trận đường chéo chứa các “giá trị riêng”.

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & & 0 \\ & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_m \end{bmatrix}$$

Phân tích giá trị riêng (ED):

- ▶ Một ma trận **vuông** M được phân rã thành $M=Q \Lambda Q^t$.
- ▶ Các cột của Q được gọi là “vector riêng”.

$$Q = \begin{bmatrix} \uparrow & \cdots & \uparrow \\ \mathbf{q}_1 & \cdots & \mathbf{q}_m \\ \downarrow & \cdots & \downarrow \end{bmatrix}$$

- ▶ Giữa một vector riêng và giá trị riêng của nó, chúng ta có mối quan hệ sau:

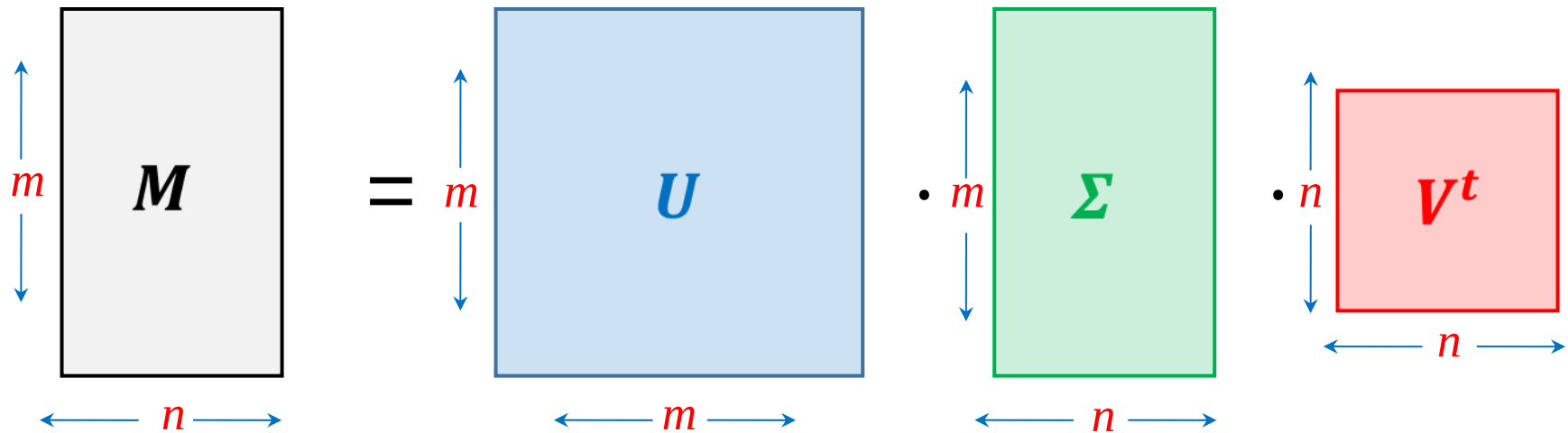
$$M\mathbf{q}_i = \lambda_i \mathbf{q}_i$$

- ▶ Giữa hai vector riêng bất kỳ, chúng ta có điều kiện trực giao sau:

$$\mathbf{q}_i \cdot \mathbf{q}_j = \delta_{ij} \quad \Leftrightarrow \quad QQ^t = Q^tQ = I$$

Phân tích giá trị đơn (SVD):

- Một ma trận M được phân tách thành $M=U\Sigma V^t$.



Phân tích giá trị đơn (SVD):

- ▶ Một ma trận M được phân tách thành $M=U\Sigma V^t$.
- ▶ Σ chứa giá trị đơn dưới dạng phân tử trên đường chéo.
- ▶ Những giá trị đơn này được sắp xếp theo thứ tự từ lớn đến nhỏ: $\sigma_1 > \sigma_2 > \dots > \sigma_m$

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \dots & 0 & 0 \\ 0 & \sigma_2 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & \sigma_m & 0 \end{bmatrix}$$

Phân tích giá trị đơn (SVD):

- ▶ Một ma trận M được phân tách thành $M=U\Sigma V^t$.
- ▶ Cột của U là “vector suy biến bên trái”.
- ▶ Cột của V là “vector suy biến bên phải”.

$$U = \begin{bmatrix} \uparrow & \dots & \uparrow \\ \mathbf{u}_1 & \dots & \mathbf{u}_m \\ \downarrow & \dots & \downarrow \end{bmatrix} \quad V = \begin{bmatrix} \uparrow & \dots & \uparrow \\ \mathbf{v}_1 & \dots & \mathbf{v}_n \\ \downarrow & \dots & \downarrow \end{bmatrix}$$

- ▶ Giữa một tập vector suy biến bên trái và bên phải cùng với giá trị suy biến của chúng, ta có:

$$M \mathbf{v}_i = \sigma_i \mathbf{u}_i$$

- ▶ Giữa hai vector suy biến bất kỳ, chúng ta có điều kiện vuông góc như sau:

$$\mathbf{v}_i \cdot \mathbf{v}_j = \delta_{ij} \Leftrightarrow VV^t = V^tV = I$$

$$\mathbf{u}_i \cdot \mathbf{u}_j = \delta_{ij} \Leftrightarrow UU^t = U^tU = I$$

■ Mọi tương quan giữa ED và SVD:

- 1) Nếu X là một ma trận được cấu thành từ các điểm được chuẩn hóa, chúng ta có thể áp dụng SVD: $X=U\Sigma V^t$.
- 2) Sử dụng dạng đã phân tích, chúng ta có thể biểu thị ma trận hiệp phương sai như sau:

$$\begin{aligned} &= U\Sigma V^t V\Sigma^t U^t \\ &= U\Sigma\Sigma^t U^t \end{aligned} \quad \left. \vphantom{\begin{aligned} &= U\Sigma V^t V\Sigma^t U^t \\ &= U\Sigma\Sigma^t U^t \end{aligned}} \right\} \text{ Theo tính trực giao } V^t V = I$$

- ▶ Có thể thấy rằng $\Sigma\Sigma^t$ là một ma trận đường chéo với các phần tử bằng σ_i^2 .
- ▶ Như vậy, $\Sigma\Sigma^t$ tương tự với ma trận giá trị riêng Λ .
- ▶ Đồng thời, có thể thấy ma trận suy biến bên trái U trái tương tự như ma trận vectơ riêng Q .

■ Mọi tương quan giữa ED và SVD:

- ▶ Hãy tổng hợp lại vào bảng sau.

ED	SVD
$X X^t = Q \Lambda Q^t$	$X X^t = U \Sigma \Sigma^t U^t$
	$\Lambda = \Sigma \Sigma^t$
	$Q = U$

Phân tích thành phần chính

| Tính toán các thành phần chính:

- Cuối cùng, hãy tổng hợp PCA theo dạng phân tích ma trận.

	ED	SVD
Đối tượng phân tích	Ma trận hiệp phương sai $\mathbf{XX}^t$	Ma trận dữ liệu $\mathbf{X}$
PCs	Các cột của ma trận $\mathbf{Q}$	Các cột của ma trận $\mathbf{U}$
σ_i^2 or σ_i	Các phần tử trên đường chéo của ma trận $\mathbf{\Lambda}$ là σ_i^2 .	Các phần tử trên đường chéo của ma trận $\mathbf{\Sigma}$ là σ_i .

Bài 4.

Mô hình nhân tố tuyến tính để giảm chiều dữ liệu

| 4.1. Phân tích thành phần chính

| 4.2. Ứng dụng của các thành phần chính

Giảm số chiều dữ liệu

■ Về việc giảm số chiều dữ liệu:

- ▶ Số lượng thành phần chính (PC) bằng với số lượng biến, chẳng hạn như k .
- ▶ Các biến ban đầu X_i có thể được biểu diễn dưới dạng PC:

$$X_i = \beta_{1,i}PC_1 + \beta_{2,i}PC_2 + \dots + \beta_{k,i}PC_k$$

- ▶ Các PC được sắp xếp theo phương sai: $\sigma_1^2 > \sigma_2^2 > \sigma_3^2 > \dots > \sigma_k^2$
- ▶ Vì vậy, chúng tôi có thể giảm số chiều bắt đầu từ PC cuối cùng, ($q < k$):

$$X_i \approx \beta_{1,i}PC_1 + \beta_{2,i}PC_2 + \dots + \beta_{q,i}PC_q \quad \text{"Giảm số chiều dữ liệu đầu vào"}$$

| Ưu điểm:

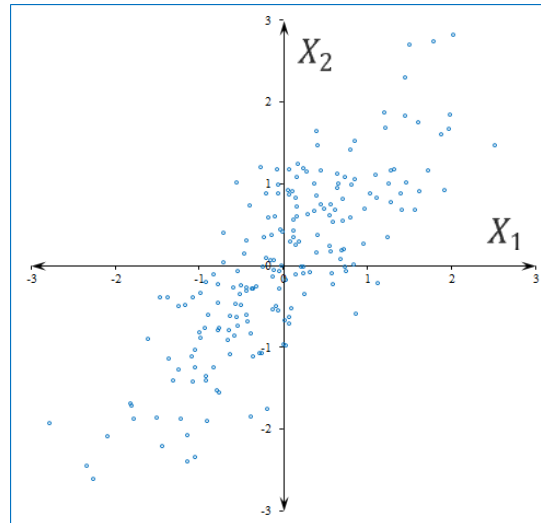
- Có thể đơn giản hóa dữ liệu và giảm thiểu lỗi quá khớp.
- Chỉ thu lại những đặc trưng nổi trội nhất.

| Nhược điểm:

- Mất dữ liệu chi tiết.
- Khó biên dịch một cách rõ ràng.

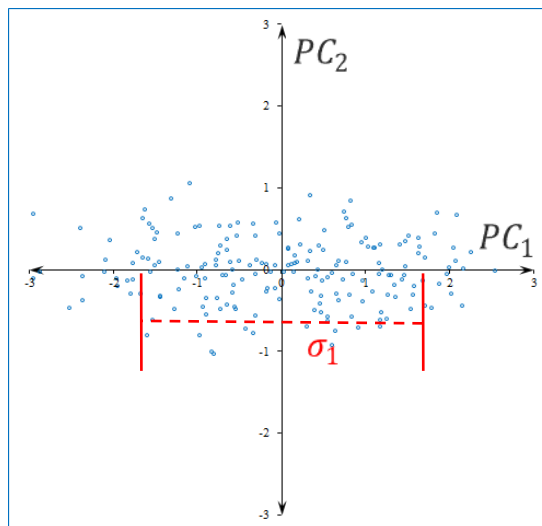
Giảm số chiều dữ liệu:

- Giả sử có một tập dữ liệu với hai biến được trực quan hóa trên một mặt phẳng:



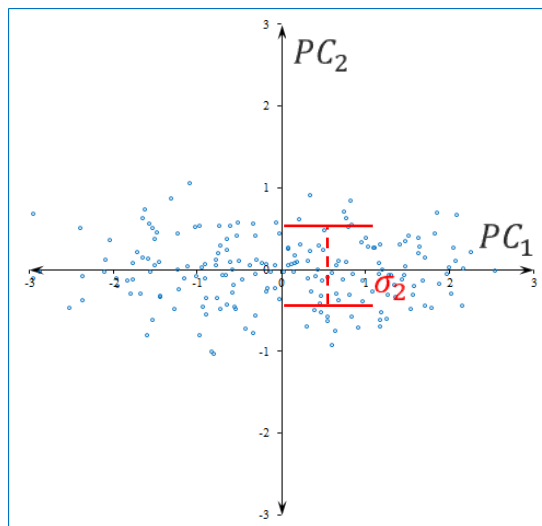
■ Giảm số chiều dữ liệu:

- ▶ Có thể biểu thị các quan sát bằng cách dùng PC_1 và PC_2 làm trục tọa độ mới:



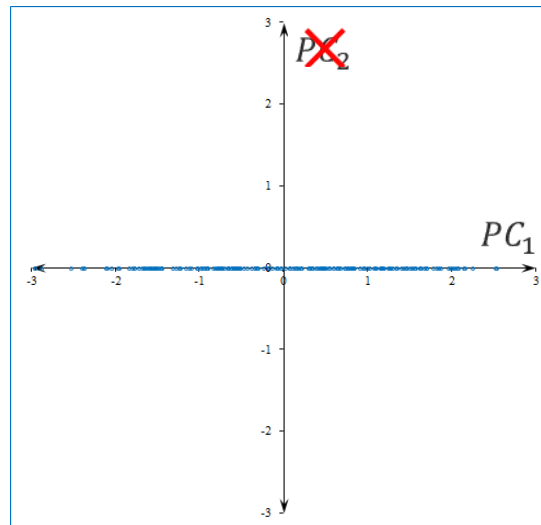
■ Giảm số chiều dữ liệu:

- ▶ Có thể biểu thị các quan sát bằng cách dùng PC_1 và PC_2 làm trục tọa độ mới:



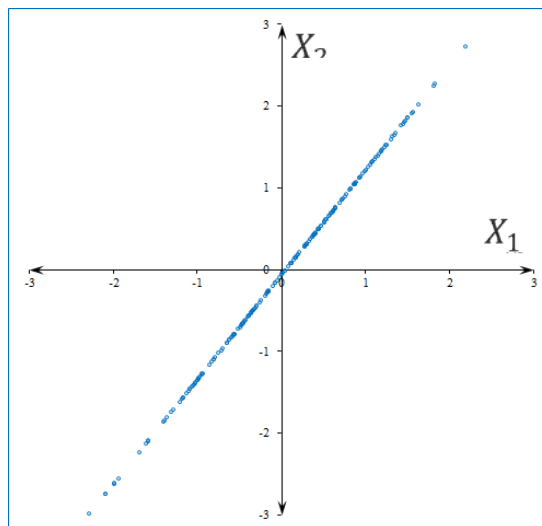
Giảm số chiều dữ liệu:

- ▶ Có thể loại bỏ các biểu tượng được biểu thị bằng PC_2 tương ứng với phương sai nhỏ hơn:



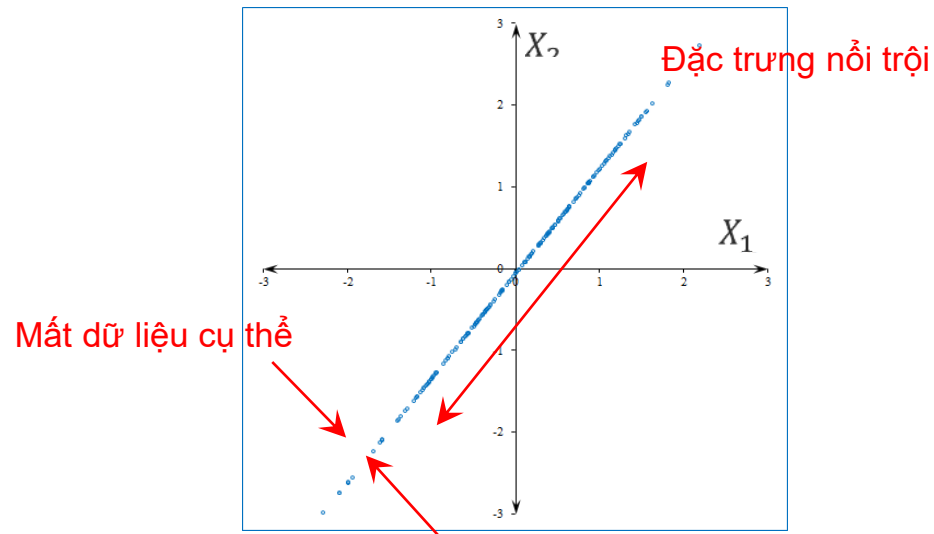
■ Giảm số chiều dữ liệu:

- ▶ Giờ chúng ta có thể **trở về** hệ tọa độ ban đầu và hiển thị “dữ liệu đầu vào với số chiều giảm”:



Giảm số chiều dữ liệu:

- Giờ chúng ta có thể **trở về** hệ tọa độ ban đầu và hiển thị “dữ liệu đầu vào với số chiều giảm”:



- Có thể thấy rằng các dữ liệu chi tiết đã bị mất và chỉ còn lại đặc trưng nổi trội nhất.

■ Giảm số chiều dữ liệu:

- Tổng phương sai là: $\sigma_{total}^2 = \sigma_1^2 + \sigma_2^2 + \sigma_3^2 + \dots + \sigma_k^2$
- Sau đó, chúng ta có thể tính tỷ số phương sai cộng dồn:

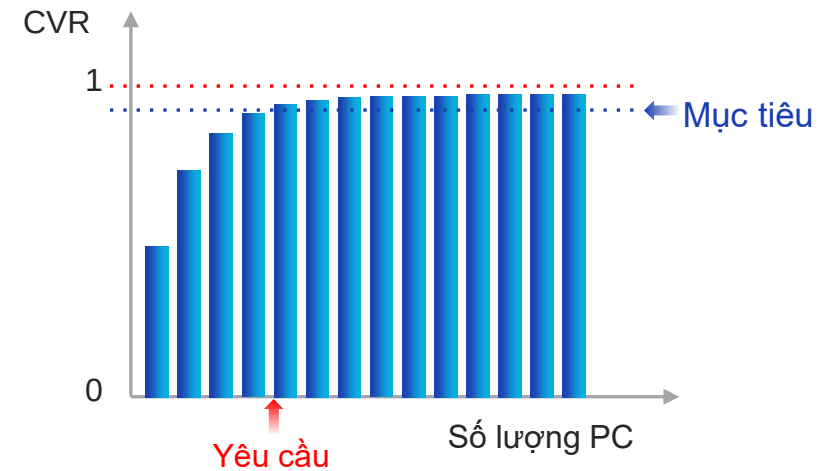
$$CVR_1 = \frac{\sigma_1^2}{\sigma_{total}^2}$$

$$CVR_2 = \frac{\sigma_1^2 + \sigma_2^2}{\sigma_{total}^2}$$

$$CVR_3 = \frac{\sigma_1^2 + \sigma_2^2 + \sigma_3^2}{\sigma_{total}^2}$$

⋮

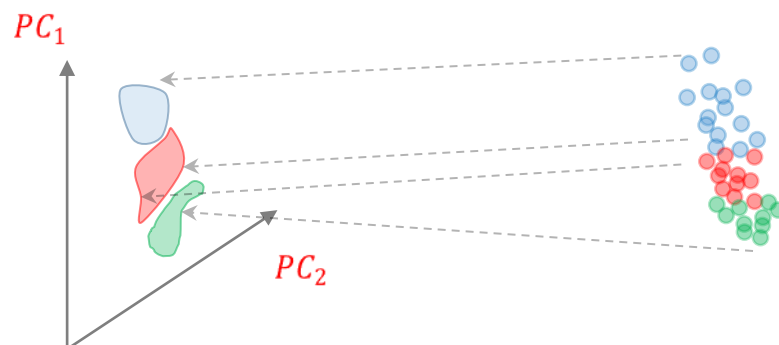
- Có thể thiết lập CVR mục tiêu và xác định số lượng PC cần thiết.



Trực quan hóa dữ liệu nhiều chiều

Trực quan hóa dữ liệu nhiều chiều:

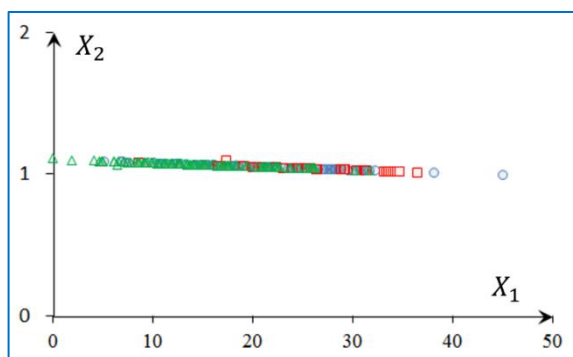
- ▶ PC_1 và PC_2 là các chiều của phương sai lớn nhất và lớn thứ hai.
- ▶ PC_1 và PC_2 xác định mặt phẳng trải rộng nhất để chiếu các tọa độ nhiều chiều.



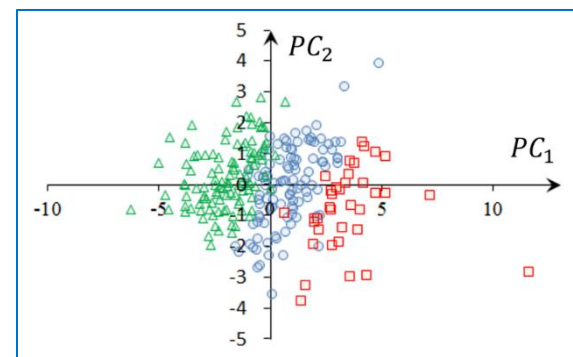
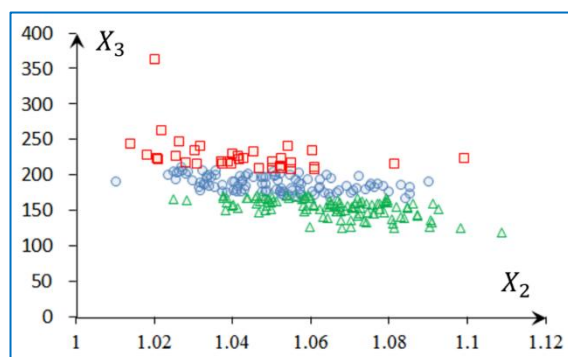
- ▶ Dễ thực thi: chỉ dùng hai thành phần đầu tiên của điểm đã biến đổi.

Trực quan hóa dữ liệu nhiều chiều:

VD



Được chiếu lên tập biến gốc



Được chiếu lên mặt phẳng do PC_1
và PC_2 xác định.

Bài tập coding số 0404



Làm theo các bước thực hành trong tệp 'ex_0404.ipynb'.

A person is sitting at a desk in a dimly lit office. They are holding a brown paper coffee cup in their left hand and a pen in their right hand, which is resting on a keyboard. On the desk, there is a laptop, a keyboard, a mouse, and some papers. The background is dark and out of focus.

End of Document



SAMSUNG

Together for Tomorrow!
Enabling People

Education for Future Generations

©2022 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.